LaBRADOR

October 29, 2026

Blockstream Research

research.blockstream.com

github.com/BlockstreamResearch/zk-book



Plan

1 Dot Product Constraint Systems

2 The Main Protocol

3 Recursion

4 Parameters and Security

5 Reduction From R1CS

What will we learn today?

- ALS20/LNP22 proofs grow **linearly** with the witness (10–20 KB for small statements). For circuits with a million gates, this is no better than sending the witness.

What will we learn today?

- ALS20/LNP22 proofs grow **linearly** with the witness (10–20 KB for small statements). For circuits with a million gates, this is no better than sending the witness.
- **LaBRADOR** (*Lattice-Based Recursively Amortized Demonstration Of R1CS*): an R1CS with 2^{20} constraints in about 58 KB, based only on Module-SIS.

What will we learn today?

- ALS20/LNP22 proofs grow **linearly** with the witness (10–20 KB for small statements). For circuits with a million gates, this is no better than sending the witness.
- **LaBRADOR** (*Lattice-Based Recursively Amortized Demonstration Of R1CS*): an R1CS with 2^{20} constraints in about 58 KB, based only on Module-SIS.
- Two ideas:
  - **amortization**: fold r secret vectors into one random linear combination;
  - **recursion** (as in Bulletproofs): the last message is a witness for a statement of the same form.

What will we learn today?

- ALS20/LNP22 proofs grow **linearly** with the witness (10–20 KB for small statements). For circuits with a million gates, this is no better than sending the witness.
- **LaBRADOR** (*Lattice-Based Recursively Amortized Demonstration Of R1CS*): an R1CS with 2^{20} constraints in about 58 KB, based only on Module-SIS.
- Two ideas:
  - **amortization**: fold r secret vectors into one random linear combination;
  - **recursion** (as in Bulletproofs): the last message is a witness for a statement of the same form.

Key idea

Whenever the last message of a proof is a solution to a system of the same type as the original one, the prover can recursively prove knowledge of it instead of sending it.

Before We Start

LaBRADOR is not zero-knowledge

This makes it noticeably simpler: no masks $\mathbf{y}$ and no rejection sampling.

Before We Start

LaBRADOR is not zero-knowledge

This makes it noticeably simpler: no masks $\mathbf{y}$ and no rejection sampling.

Notation

- $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^d + 1)$, where $T^d + 1$ splits into two irreducible factors modulo q (as in LNP22).
- $f^* := f(T^{-1})$ is the conjugate, $\mathrm{ct}(f)$ the constant coefficient.
- $\|\mathbf{f}\|_2$ is the ℓ^2 norm of the coefficient vector of $\mathbf{f} \in \mathcal{R}_q^n$; $\vec{\mathbf{f}} \in \mathbb{Z}_q^{nd}$ is the coefficient vector itself.

Before We Start

LaBRADOR is not zero-knowledge

This makes it noticeably simpler: no masks $\mathbf{y}$ and no rejection sampling.

Notation

- $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^d + 1)$, where $T^d + 1$ splits into two irreducible factors modulo q (as in LNP22).
- $f^* := f(T^{-1})$ is the conjugate, $\mathrm{ct}(f)$ the constant coefficient.
- $\|\mathbf{f}\|_2$ is the ℓ^2 norm of the coefficient vector of $\mathbf{f} \in \mathcal{R}_q^n$; $\vec{\mathbf{f}} \in \mathbb{Z}_q^{nd}$ is the coefficient vector itself.

Plan: define dot product constraint systems → build the main protocol (Attempts #1–#4) → recursion → parameters → reduction from R1CS.

Dot Product Constraint Systems

Dot Product Functions

Requirements for the relation:

1. expressive enough to encode R1CS;
2. the verifier's final checks form a statement of the **same type** (recursion).

Dot Product Functions

Requirements for the relation:

1. expressive enough to encode R1CS;
2. the verifier's final checks form a statement of the **same type** (recursion).

For $\mathbf{a}, \mathbf{b} \in \mathcal{R}_q^n$, $\langle \mathbf{a}, \mathbf{b} \rangle := \sum_{i \in [n]} a_i b_i \in \mathcal{R}_q$. A **dot product function** of $\mathbf{s}_1, \ldots, \mathbf{s}_r \in \mathcal{R}_q^n$ is

$$f(\mathbf{s}_1, \ldots, \mathbf{s}_r) = \sum_{i,j \in [r]} a_{ij} \langle \mathbf{s}_i, \mathbf{s}_j \rangle + \sum_{i \in [r]} \langle \boldsymbol{\varphi}_i, \mathbf{s}_i \rangle + a_0,$$

where $a_{ij}, a_0 \in \mathcal{R}_q$ and $\boldsymbol{\varphi}_i \in \mathcal{R}_q^n$ are public; w.l.o.g. $\mathbf{A} := (a_{ij})_{i,j}$ is symmetric.

Dot Product Functions

Requirements for the relation:

1. expressive enough to encode R1CS;
2. the verifier's final checks form a statement of the **same type** (recursion).

For $\mathbf{a}, \mathbf{b} \in \mathcal{R}_q^n$, $\langle \mathbf{a}, \mathbf{b} \rangle := \sum_{i \in [n]} a_i b_i \in \mathcal{R}_q$. A **dot product function** of $\mathbf{s}_1, \ldots, \mathbf{s}_r \in \mathcal{R}_q^n$ is

$$f(\mathbf{s}_1, \ldots, \mathbf{s}_r) = \sum_{i,j \in [r]} a_{ij} \langle \mathbf{s}_i, \mathbf{s}_j \rangle + \sum_{i \in [r]} \langle \boldsymbol{\varphi}_i, \mathbf{s}_i \rangle + a_0,$$

where $a_{ij}, a_0 \in \mathcal{R}_q$ and $\boldsymbol{\varphi}_i \in \mathcal{R}_q^n$ are public; w.l.o.g. $\mathbf{A} := (a_{ij})_{i,j}$ is symmetric.

Remark

A less general version of the quadratic forms $\mathbf{s}^\top \mathbf{Q} \mathbf{s} + \mathbf{p}^\top \mathbf{s} + p_0$ from LNP22. We denote this class of functions by $\mathcal{Q}$.

The Principal Relation

Definition (Principal Relation)

Fix the *rank* $n \geq 1$, the *multiplicity* $r \geq 1$, and a norm bound $\beta > 0$. The **principal relation** of LaBRADOR is

$$\mathcal{R}_{\mathbf{dot}} := \left\{ \begin{array}{l} \mathbb{x} = (\mathcal{F}, \mathcal{F}_{\mathsf{ct}}, \beta) \\ \mathbb{w} = (\mathbf{s}_1, \ldots, \mathbf{s}_r) \end{array} \middle| \begin{array}{l} f(\mathbf{s}_1, \ldots, \mathbf{s}_r) = 0 \text{ for all } f \in \mathcal{F}, \\ \mathsf{ct}(g(\mathsf{s}_1, \ldots, \mathsf{s}_r)) = 0 \text{ for all } g \in \mathcal{F}_{\mathsf{ct}}, \\ \sum_{i \in [r]} \|\mathbf{s}_i\|_2^2 \leq \beta^2 \end{array} \right\},$$

where $\mathcal{F}, \mathcal{F}_{\mathsf{ct}} \subseteq Q$ and $\mathbf{s}_1, \ldots, \mathbf{s}_r \in \mathcal{R}_q^n$.

The Principal Relation

Definition (Principal Relation)

Fix the *rank* $n \geq 1$, the *multiplicity* $r \geq 1$, and a norm bound $\beta > 0$. The **principal relation** of LaBRADOR is

$$\mathcal{R}_{\mathbf{dot}} := \left\{ \begin{array}{l} \mathbb{x} = (\mathcal{F}, \mathcal{F}_{\mathsf{ct}}, \beta) \\ \mathbb{w} = (\mathbf{s}_1, \ldots, \mathbf{s}_r) \end{array} \middle| \begin{array}{l} f(\mathbf{s}_1, \ldots, \mathbf{s}_r) = 0 \text{ for all } f \in \mathcal{F}, \\ \mathsf{ct}(g(\mathsf{s}_1, \ldots, \mathsf{s}_r)) = 0 \text{ for all } g \in \mathcal{F}_{\mathsf{ct}}, \\ \sum_{i \in [r]} \|\mathbf{s}_i\|_2^2 \leq \beta^2 \end{array} \right\},$$

where $\mathcal{F}, \mathcal{F}_{\mathsf{ct}} \subseteq Q$ and $\mathbf{s}_1, \ldots, \mathbf{s}_r \in \mathcal{R}_q^n$.

- $\mathcal{F}$: constraints that vanish as **whole polynomials**.
- $\mathcal{F}_{\mathsf{ct}}$: only the **constant coefficient** must vanish; by the constant coefficient lemma, these are linear and quadratic relations over $\mathbb{Z}_q$ among the coefficients.

Examples

Example

Statements on a single vector $\mathbf{s} \in \mathcal{R}_q^n$ $(r = 1)$:

1. *Opening of an Ajtai commitment.* $\mathbf{t} = \mathbf{A}\mathbf{s}$ for $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$ with rows $\mathbf{a}_1^\top, \ldots, \mathbf{a}_\kappa^\top$ is the collection of κ linear functions $f_k(\mathbf{s}) = \langle \mathbf{a}_k, \mathbf{s} \rangle - t_k$ in $\mathcal{F}$.

Examples

Example

Statements on a single vector $\mathbf{s} \in \mathcal{R}_q^n$ $(r = 1)$:

1. *Opening of an Ajtai commitment.* $\mathbf{t} = \mathbf{A}\mathbf{s}$ for $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$ with rows $\mathbf{a}_1^\top, \ldots, \mathbf{a}_\kappa^\top$ is the collection of κ linear functions $f_k(\mathbf{s}) = \langle \mathbf{a}_k, \mathbf{s} \rangle - t_k$ in $\mathcal{F}$.

2. *A single coefficient.* The j-th coefficient of the i-th entry of $\mathbf{s}$ equals $\mathrm{ct}(\langle (T^j \mathbf{e}_i)^*, \mathbf{s} \rangle)$. "This coefficient equals 5" is $g(\mathbf{s}) = \langle (T^j \mathbf{e}_i)^*, \mathbf{s} \rangle - 5$ in $\mathcal{F}_{\mathsf{ct}}$.

Examples

Example

Statements on a single vector $\mathbf{s} \in \mathcal{R}_q^n$ $(r = 1)$:

1. *Opening of an Ajtai commitment.* $\mathbf{t} = \mathbf{A}\mathbf{s}$ for $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$ with rows $\mathbf{a}_1^\top, \ldots, \mathbf{a}_\kappa^\top$ is the collection of κ linear functions $f_k(\mathbf{s}) = \langle \mathbf{a}_k, \mathbf{s} \rangle - t_k$ in $\mathcal{F}$.

2. *A single coefficient.* The j-th coefficient of the i-th entry of $\mathbf{s}$ equals $\mathrm{ct}(\langle (T^j \mathbf{e}_i)^*, \mathbf{s} \rangle)$. "This coefficient equals 5" is $g(\mathbf{s}) = \langle (T^j \mathbf{e}_i)^*, \mathbf{s} \rangle - 5$ in $\mathcal{F}_{\mathrm{ct}}$.

3. *Norm.* $\|\mathbf{s}\|_2^2 = \mathrm{ct}(\langle \mathbf{s}^*, \mathbf{s} \rangle)$ is not a dot product function of $\mathbf{s}$ alone (the conjugate appears). Fix: add $\mathbf{s}^*$ to the witness (later). The bound β is already part of the relation.

Examples

Example

Statements on a single vector $\mathbf{s} \in \mathcal{R}_q^n$ $(r = 1)$:

1. *Opening of an Ajtai commitment.* $\mathbf{t} = \mathbf{A}\mathbf{s}$ for $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$ with rows $\mathbf{a}_1^\top, \ldots, \mathbf{a}_\kappa^\top$ is the collection of κ linear functions $f_k(\mathbf{s}) = \langle \mathbf{a}_k, \mathbf{s} \rangle - t_k$ in $\mathcal{F}$.

2. *A single coefficient.* The j-th coefficient of the i-th entry of $\mathbf{s}$ equals $\mathrm{ct}(\langle (T^j \mathbf{e}_i)^*, \mathbf{s} \rangle)$. "This coefficient equals 5" is $g(\mathbf{s}) = \langle (T^j \mathbf{e}_i)^*, \mathbf{s} \rangle - 5$ in $\mathcal{F}_{\mathsf{ct}}$.

3. *Norm.* $\|\mathbf{s}\|_2^2 = \mathrm{ct}(\langle \mathbf{s}^*, \mathbf{s} \rangle)$ is not a dot product function of $\mathbf{s}$ alone (the conjugate appears). Fix: add $\mathbf{s}^*$ to the witness (later). The bound β is already part of the relation.

Question #1

Write "I know $(\mathbf{s}, \mathbf{e})$ with $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$ and $\|\mathbf{s}\|_2^2 + \|\mathbf{e}\|_2^2 \leq \beta^2$" (a Module-LWE key) as an instance of $\mathcal{R}_{\mathsf{dot}}$ with $r = 1$.

The Main Protocol

Attempt #1: Amortized Opening

The trivial protocol sends $\mathbf{s}_1, \ldots, \mathbf{s}_r$: rn ring elements. Commit $\mathbf{t}_i = \mathbf{A}\mathbf{s}_i$ for $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$. No zero-knowledge $\Rightarrow$ drop the mask; one response for all commitments:

$$\mathbf{z} := c_1\mathbf{s}_1 + \cdots + c_r\mathbf{s}_r.$$

Attempt #1: Amortized Opening

The trivial protocol sends $\mathbf{s}_1, \ldots, \mathbf{s}_r$: rn ring elements. Commit $\mathbf{t}_i = \mathbf{A}\mathbf{s}_i$ for $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$. No zero-knowledge $\Rightarrow$ drop the mask; one response for all commitments:

$$\mathbf{z} := c_1\mathbf{s}_1 + \cdots + c_r\mathbf{s}_r.$$

$\mathcal{P}(\mathbb{x} = (\mathbf{t}_i)_{i\in[r]}, \mathbb{w} = (\mathbf{s}_i)_{i\in[r]})$:

1: **receive** $c_1, \ldots, c_r$;
2: *// amortized opening*
send $\mathbf{z} \leftarrow \sum_{i\in[r]} c_i\mathbf{s}_i$.

$\mathcal{V}(\mathbb{x} = (\mathbf{t}_i)_{i\in[r]}) \to \{0, 1\}$:

1: **send** $c_1, \ldots, c_r \leftarrow\$ C$;
2: **receive** $\mathbf{z}$;
3: **assert** $\|\mathbf{z}\|_2 \le \gamma$;
4: **return** $[\![\mathbf{A}\mathbf{z} = \sum_{i\in[r]} c_i\mathbf{t}_i]\!]$.

Attempt #1: Amortized Opening

The trivial protocol sends $\mathbf{s}_1, \ldots, \mathbf{s}_r$: rn ring elements. Commit $\mathbf{t}_i = \mathbf{A}\mathbf{s}_i$ for $\mathbf{A} \in \mathcal{R}_q^{\kappa \times n}$. No zero-knowledge $\Rightarrow$ drop the mask; one response for all commitments:

$$\mathbf{z} := c_1\mathbf{s}_1 + \cdots + c_r\mathbf{s}_r.$$

$\mathcal{P}(\mathbb{x} = (\mathbf{t}_i)_{i\in[r]}, \mathbb{w} = (\mathbf{s}_i)_{i\in[r]})$:

1: **receive** $c_1, \ldots, c_r$;
2: *// amortized opening*
send $\mathbf{z} \leftarrow \sum_{i\in[r]} c_i\mathbf{s}_i$.

$\mathcal{V}(\mathbb{x} = (\mathbf{t}_i)_{i\in[r]}) \to \{0, 1\}$:

1: **send** $c_1, \ldots, c_r \leftarrow\$ C$;
2: **receive** $\mathbf{z}$;
3: **assert** $\|\mathbf{z}\|_2 \le \gamma$;
4: **return** $[\![\mathbf{A}\mathbf{z} = \sum_{i\in[r]} c_i\mathbf{t}_i]\!]$.

- Communication: $r\kappa + n$ instead of rn ring elements (κ is a few dozen).
- Price: coefficients of $\mathbf{z}$ grow by $\approx \sqrt{r\tau}$, where $\tau := \max_{c\in C} \|c\|_2^2$.

The Challenge Set

The extractor gets a *product* of an extracted vector with a challenge difference; bounding it needs the **operator norm**

$$\|c\|_{\mathsf{op}} := \sup_{f \in \mathcal{R} \setminus \{0\}} \frac{\|cf\|_2}{\|f\|_2}.$$

The Challenge Set

The extractor gets a *product* of an extracted vector with a challenge difference; bounding it needs the **operator norm**

$$\|c\|_{\mathsf{op}} := \sup_{f \in \mathcal{R} \setminus \{0\}} \frac{\|cf\|_2}{\|f\|_2}.$$

LaBRADOR requires: $c - c'$ invertible in $\mathcal{R}_q$ for distinct challenges, $\|c\|_2^2 \leq \tau$, and $\|c\|_{\mathsf{op}} \leq \theta$.

The Challenge Set

The extractor gets a *product* of an extracted vector with a challenge difference; bounding it needs the **operator norm**

$$\|c\|_{\mathsf{op}} := \sup_{f \in \mathcal{R}\setminus\{0\}} \frac{\|cf\|_2}{\|f\|_2}.$$

LaBRADOR requires: $c - c'$ invertible in $\mathcal{R}_q$ for distinct challenges, $\|c\|_2^2 \le \tau$, and $\|c\|_{\mathsf{op}} \le \theta$.

With $N_i(c) := \#\{j : |c_j| = i\}$ and $k_0 + k_1 + k_2 = d$:

$$\mathcal{C} := \{c \in \mathcal{R} : N_0(c) = k_0,\ N_1(c) = k_1,\ N_2(c) = k_2,\ \|c\|_{\mathsf{op}} \le \theta\},$$

with $d = 64$, $(k_0, k_1, k_2) = (23, 31, 10)$, $\theta = 15$ (≈ 6 candidates per challenge).

The Challenge Set

The extractor gets a *product* of an extracted vector with a challenge difference; bounding it needs the **operator norm**

$$\|c\|_{\mathsf{op}} := \sup_{f \in \mathcal{R} \setminus \{0\}} \frac{\|cf\|_2}{\|f\|_2}.$$

LaBRADOR requires: $c - c'$ invertible in $\mathcal{R}_q$ for distinct challenges, $\|c\|_2^2 \leq \tau$, and $\|c\|_{\mathsf{op}} \leq \theta$.

With $N_i(c) := \#\{j : |c_j| = i\}$ and $k_0 + k_1 + k_2 = d$:

$$C := \{c \in \mathcal{R} : N_0(c) = k_0,\ N_1(c) = k_1,\ N_2(c) = k_2,\ \|c\|_{\mathsf{op}} \leq \theta\},$$

with $d = 64$, $(k_0, k_1, k_2) = (23, 31, 10)$, $\theta = 15$ (≈ 6 candidates per challenge).

Question #2

Express $\tau = \|c\|_2^2$ (for $c \in C$) and $\#C$ in terms of d, k_0, k_1, k_2.

Soundness and Weak Openings

As with the Lyubashevsky protocol, the extractor does not obtain the $\mathbf{s}_i$ themselves.

Definition (Weak opening)

Vectors $\mathbf{s}_i^*$ with $\mathbf{A}\mathbf{s}_i^* = \mathbf{t}_i$, together with a challenge difference $\bar{c} = c - c'$ such that $\bar{c}\mathbf{s}_i^*$ is short.

Soundness and Weak Openings

As with the Lyubashevsky protocol, the extractor does not obtain the $\mathbf{s}_i$ themselves.

Definition (Weak opening)

Vectors $\mathbf{s}_i^*$ with $\mathbf{A}\mathbf{s}_i^* = \mathbf{t}_i$, together with a challenge difference $\bar{c} = c - c'$ such that $\bar{c}\mathbf{s}_i^*$ is short.

- The commitment stays binding for weak openings under Module-SIS, with a norm bound larger by a factor proportional to θ.

Soundness and Weak Openings

As with the Lyubashevsky protocol, the extractor does not obtain the $\mathbf{s}_i$ themselves.

Definition (Weak opening)

Vectors $\mathbf{s}_i^*$ with $\mathbf{A}\mathbf{s}_i^* = \mathbf{t}_i$, together with a challenge difference $\bar{c} = c - c'$ such that $\bar{c}\mathbf{s}_i^*$ is short.

- The commitment stays binding for weak openings under Module-SIS, with a norm bound larger by a factor proportional to θ.
- Two weak openings $(\mathbf{s}^*, \bar{c})$, $(\mathbf{s}'^*, \bar{c}')$ of the same commitment: $\bar{c}'(\bar{c}\mathbf{s}^*) - \bar{c}(\bar{c}'\mathbf{s}'^*)$ lies in the kernel of $\mathbf{A}$, and its norm is bounded using the operator norms.

Soundness and Weak Openings

As with the Lyubashevsky protocol, the extractor does not obtain the $\mathbf{s}_i$ themselves.

Definition (Weak opening)

Vectors $\mathbf{s}_i^*$ with $\mathbf{A}\mathbf{s}_i^* = \mathbf{t}_i$, together with a challenge difference $\bar{c} = c - c'$ such that $\bar{c}\mathbf{s}_i^*$ is short.

- The commitment stays binding for weak openings under Module-SIS, with a norm bound larger by a factor proportional to θ.
- Two weak openings $(\mathbf{s}^*, \bar{c})$, $(\mathbf{s}'^*, \bar{c}')$ of the same commitment: $\bar{c}'(\bar{c}\mathbf{s}^*) - \bar{c}(\bar{c}'\mathbf{s}'^*)$ lies in the kernel of $\mathbf{A}$, and its norm is bounded using the operator norms.

That is why the operator norm of the challenges enters the parameters.

Attempt #2: Garbage Polynomials

Goal: show $F(\mathbf{s}_1, \ldots, \mathbf{s}_r) = 0$ for a single dot product function F. As in ALS20, evaluate on $\mathbf{z} = \sum_i c_i \mathbf{s}_i$:

$$\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j \in [r]} g_{ij}\, c_i c_j, \qquad \sum_{i \in [r]} \langle \boldsymbol{\varphi}_i, \mathbf{z} \rangle c_i = \sum_{i,j \in [r]} h_{ij}\, c_i c_j,$$

Attempt #2: Garbage Polynomials

Goal: show $F(\mathbf{s}_1, \ldots, \mathbf{s}_r) = 0$ for a single dot product function F. As in ALS20, evaluate on $\mathbf{z} = \sum_i c_i \mathbf{s}_i$:

$$\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j \in [r]} g_{ij}\, c_i c_j, \qquad \sum_{i \in [r]} \langle \boldsymbol{\varphi}_i, \mathbf{z} \rangle c_i = \sum_{i,j \in [r]} h_{ij}\, c_i c_j,$$

where the **garbage polynomials** are

$$g_{ij} := \langle \mathbf{s}_i, \mathbf{s}_j \rangle, \qquad h_{ij} := \frac{1}{2}(\langle \boldsymbol{\varphi}_i, \mathbf{s}_j \rangle + \langle \boldsymbol{\varphi}_j, \mathbf{s}_i \rangle).$$

Both matrices are symmetric and **do not depend on the challenges**.

Attempt #2: Garbage Polynomials

Goal: show $F(\mathbf{s}_1, \ldots, \mathbf{s}_r) = 0$ for a single dot product function F. As in ALS20, evaluate on $\mathbf{z} = \sum_i c_i \mathbf{s}_i$:

$$\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j \in [r]} g_{ij}\, c_i c_j, \qquad \sum_{i \in [r]} \langle \boldsymbol{\varphi}_i, \mathbf{z} \rangle c_i = \sum_{i,j \in [r]} h_{ij}\, c_i c_j,$$

where the **garbage polynomials** are

$$g_{ij} := \langle \mathbf{s}_i, \mathbf{s}_j \rangle, \qquad h_{ij} := \frac{1}{2}(\langle \boldsymbol{\varphi}_i, \mathbf{s}_j \rangle + \langle \boldsymbol{\varphi}_j, \mathbf{s}_i \rangle).$$

Both matrices are symmetric and **do not depend on the challenges**.

Moreover, the constraint is a linear function of the garbage:

$$F(\mathbf{s}_1, \ldots, \mathbf{s}_r) = \sum_{i,j \in [r]} a_{ij} g_{ij} + \sum_{i \in [r]} h_{ii} + a_0.$$

Attempt #2: Garbage Polynomials

Goal: show $F(\mathbf{s}_1, \ldots, \mathbf{s}_r) = 0$ for a single dot product function F. As in ALS20, evaluate on $\mathbf{z} = \sum_i c_i \mathbf{s}_i$:

$$\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j \in [r]} g_{ij}\, c_i c_j, \qquad \sum_{i \in [r]} \langle \boldsymbol{\varphi}_i, \mathbf{z} \rangle c_i = \sum_{i,j \in [r]} h_{ij}\, c_i c_j,$$

where the **garbage polynomials** are

$$g_{ij} := \langle \mathbf{s}_i, \mathbf{s}_j \rangle, \qquad h_{ij} := \frac{1}{2}(\langle \boldsymbol{\varphi}_i, \mathbf{s}_j \rangle + \langle \boldsymbol{\varphi}_j, \mathbf{s}_i \rangle).$$

Both matrices are symmetric and **do not depend on the challenges**.

Moreover, the constraint is a linear function of the garbage:

$$F(\mathbf{s}_1, \ldots, \mathbf{s}_r) = \sum_{i,j \in [r]} a_{ij} g_{ij} + \sum_{i \in [r]} h_{ii} + a_0.$$

So: send the garbage together with $\mathbf{z}$, and check the three equations.

Attempt #2: the Protocol

$\mathcal{P}(\mathbb{x}, \mathbb{w} = (\mathbf{s}_i)_{i\in[r]})$:

1: $g_{ij} \leftarrow \langle \mathbf{s}_i, \mathbf{s}_j \rangle$ for $i \le j$;
2: $h_{ij} \leftarrow \frac{1}{2}(\langle \boldsymbol{\varphi}_i, \mathbf{s}_j \rangle + \langle \boldsymbol{\varphi}_j, \mathbf{s}_i \rangle)$ for $i \le j$;
3: **send** $(g_{ij}, h_{ij})_{i\le j}$;
4: **receive** $c_1, \ldots, c_r$;
5: **send** $\mathbf{z} \leftarrow \sum_{i\in[r]} c_i \mathbf{s}_i$.

$\mathcal{V}(\mathbb{x} = ((\mathbf{t}_i)_{i\in[r]}, F)) \to \{0, 1\}$:

1: **receive** $(g_{ij}, h_{ij})_{i\le j}$;
2: **send** $c_1, \ldots, c_r \leftarrow\!\$\ C$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \le \gamma$;
5: **assert** $\mathbf{A}\mathbf{z} = \sum_i c_i \mathbf{t}_i$;
6: **assert** $\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j} g_{ij} c_i c_j$;
7: **assert** $\sum_i \langle \boldsymbol{\varphi}_i, \mathbf{z} \rangle c_i = \sum_{i,j} h_{ij} c_i c_j$;
8: **return** $[\![\sum_{i,j} a_{ij} g_{ij} + \sum_i h_{ii} + a_0 = 0]\!]$.

Attempt #2: the Protocol

$\mathcal{P}(\mathbb{x}, \mathbb{w} = (\mathbf{s}_i)_{i\in[r]})$:

1: $g_{ij} \leftarrow \langle \mathbf{s}_i, \mathbf{s}_j \rangle$ for $i \le j$;
2: $h_{ij} \leftarrow \frac{1}{2}(\langle \boldsymbol{\varphi}_i, \mathbf{s}_j \rangle + \langle \boldsymbol{\varphi}_j, \mathbf{s}_i \rangle)$ for $i \le j$;
3: **send** $(g_{ij}, h_{ij})_{i\le j}$;
4: **receive** $c_1, \ldots, c_r$;
5: **send** $\mathbf{z} \leftarrow \sum_{i\in[r]} c_i \mathbf{s}_i$.

$\mathcal{V}(\mathbb{x} = ((\mathbf{t}_i)_{i\in[r]}, F)) \to \{0, 1\}$:

1: **receive** $(g_{ij}, h_{ij})_{i\le j}$;
2: **send** $c_1, \ldots, c_r \leftarrow\$ C$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \le \gamma$;
5: **assert** $\mathbf{A}\mathbf{z} = \sum_i c_i \mathbf{t}_i$;
6: **assert** $\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j} g_{ij} c_i c_j$;
7: **assert** $\sum_i \langle \boldsymbol{\varphi}_i, \mathbf{z} \rangle c_i = \sum_{i,j} h_{ij} c_i c_j$;
8: **return** $[\![\sum_{i,j} a_{ij} g_{ij} + \sum_i h_{ii} + a_0 = 0]\!]$.

Why sound? The garbage is sent *before* the challenges: a lie makes the two sides of a check different quadratic polynomials in $c_1, \ldots, c_r$, agreeing on random challenges with negligible probability (Schwartz–Zippel over $\mathcal{R}_q$).

Attempt #2: the Protocol

$\mathcal{P}(\mathbb{x}, \mathbb{w} = (\mathbf{s}_i)_{i\in[r]})$:

1: $g_{ij} \leftarrow \langle \mathbf{s}_i, \mathbf{s}_j \rangle$ for $i \le j$;
2: $h_{ij} \leftarrow \frac{1}{2}(\langle \boldsymbol{\varphi}_i, \mathbf{s}_j \rangle + \langle \boldsymbol{\varphi}_j, \mathbf{s}_i \rangle)$ for $i \le j$;
3: **send** $(g_{ij}, h_{ij})_{i\le j}$;
4: **receive** $c_1, \ldots, c_r$;
5: **send** $\mathbf{z} \leftarrow \sum_{i\in[r]} c_i \mathbf{s}_i$.

$\mathcal{V}(\mathbb{x} = ((\mathbf{t}_i)_{i\in[r]}, F)) \to \{0, 1\}$:

1: **receive** $(g_{ij}, h_{ij})_{i\le j}$;
2: **send** $c_1, \ldots, c_r \leftarrow\$ C$;
3: **receive** $\mathbf{z}$;
4: **assert** $\|\mathbf{z}\|_2 \le \gamma$;
5: **assert** $\mathbf{A}\mathbf{z} = \sum_i c_i \mathbf{t}_i$;
6: **assert** $\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j} g_{ij} c_i c_j$;
7: **assert** $\sum_i \langle \boldsymbol{\varphi}_i, \mathbf{z} \rangle c_i = \sum_{i,j} h_{ij} c_i c_j$;
8: **return** $[\![\sum_{i,j} a_{ij} g_{ij} + \sum_i h_{ii} + a_0 = 0]\!]$.

Why sound? The garbage is sent *before* the challenges: a lie makes the two sides of a check different quadratic polynomials in $c_1, \ldots, c_r$, agreeing on random challenges with negligible probability (Schwartz–Zippel over $\mathcal{R}_q$).

Price: $r^2 + r$ garbage polynomials, quadratic in r (we return to this later).

Attempt #3: Aggregating Constraints

Aggregating $\mathcal{F}$. For $f_1, \ldots, f_K \in \mathcal{F}$, the verifier sends $\alpha_1, \ldots, \alpha_K \leftarrow\$ \mathcal{R}_q$, and the prover proves that $F := \sum_{k \in [K]} \alpha_k f_k$ vanishes (again a dot product function).

Attempt #3: Aggregating Constraints

Aggregating $\mathcal{F}$. For $f_1, \ldots, f_K \in \mathcal{F}$, the verifier sends $\alpha_1, \ldots, \alpha_K \leftarrow\$ \mathcal{R}_q$, and the prover proves that $F := \sum_{k\in[K]} \alpha_k f_k$ vanishes (again a dot product function).

Aggregating $\mathcal{F}_{\mathsf{ct}}$. Coefficients must come from $\mathbb{Z}_q$ (multiplying by a general element of $\mathcal{R}_q$ may break $\mathsf{ct} = 0$). One combination has error $1/q$, so take $K_{\mathsf{ct}} := \lceil \lambda / \log q \rceil$ combinations with $\boldsymbol{\psi}_1, \ldots, \boldsymbol{\psi}_{K_{\mathsf{ct}}} \leftarrow\$ \mathbb{Z}_q^L$:

$$G_k := \sum_{\ell\in[L]} \psi_{k,\ell} g_\ell, \quad k \in [K_{\mathsf{ct}}].$$

For $q \approx 2^{32}$, only $K_{\mathsf{ct}} = 4$.

Attempt #3: Constant Coefficients without Masking

No masking needed

Write $G_k = Q_k + b_k$. LNP22 had to mask the non-constant coefficients; LaBRADOR sends $v_k := Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r)$ in the clear. The verifier checks $\mathrm{ct}(v_k) = -\mathrm{ct}(b_k)$, and

$$Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r) - v_k = 0$$

is an *ordinary* constraint added to $\mathcal{F}$ before aggregating it.

Attempt #3: Constant Coefficients without Masking

No masking needed

Write $G_k = Q_k + b_k$. LNP22 had to mask the non-constant coefficients; LaBRADOR sends $v_k := Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r)$ in the clear. The verifier checks $\mathrm{ct}(v_k) = -\mathrm{ct}(b_k)$, and

$$Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r) - v_k = 0$$

is an *ordinary* constraint added to $\mathcal{F}$ before aggregating it.

Result

The whole statement collapses into a single function F, proven with Attempt #2 at the cost of only K_{ct} additional polynomials v_k.

Attempt #4: Johnson–Lindenstrauss Projection

$\|\mathbf{z}\|_2 \leq \gamma$ only yields weak openings (norm may exceed β a lot); the R1CS reduction needs a small slack.

Attempt #4: Johnson–Lindenstrauss Projection

$\|\mathbf{z}\|_2 \leq \gamma$ only yields weak openings (norm may exceed β a lot); the R1CS reduction needs a small slack.

With $\mathcal{B}$ on $\{0, \pm 1\}$, $\Pr[0] = 1/2$, $\Pr[\pm 1] = 1/4$: the verifier sends $\mathbf{\Pi}_1, \ldots, \mathbf{\Pi}_r \leftarrow\$ \mathcal{B}^{256 \times nd}$, the prover sends

$$\mathbf{p} := \sum_{i \in [r]} \mathbf{\Pi}_i \vec{\mathbf{s}}_i \in \mathbb{Z}_q^{256},$$

and the verifier checks that $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$.

Attempt #4: Johnson–Lindenstrauss Projection

$\|\mathbf{z}\|_2 \le \gamma$ only yields weak openings (norm may exceed β a lot); the R1CS reduction needs a small slack.

With $\mathcal{B}$ on $\{0, \pm 1\}$, $\Pr[0] = 1/2$, $\Pr[\pm 1] = 1/4$: the verifier sends $\mathbf{\Pi}_1, \ldots, \mathbf{\Pi}_r \leftarrow\$ \mathcal{B}^{256 \times nd}$, the prover sends

$$\mathbf{p} := \sum_{i \in [r]} \mathbf{\Pi}_i \vec{\mathbf{s}}_i \in \mathbb{Z}_q^{256},$$

and the verifier checks that $\|\mathbf{p}\|_2 \le \sqrt{128}\beta$.

Lemma

Let $b \le q/125$ and let $\mathbf{w} \in \mathbb{Z}_q^m$ be any vector with $\|\mathbf{w}\|_2 \ge b$. Then,

$$\Pr_{\mathbf{\Pi} \leftarrow\$ \mathcal{B}^{256 \times m}} \left[\|\mathbf{\Pi}\mathbf{w} \bmod q\|_2 < \sqrt{30}\, b \right] \lesssim 2^{-128}.$$

A projection of a long vector is long, even after the reduction modulo q.

Attempt #4: Analysis

- Honest case: $\mathbb{E}[\|\mathbf{\Pi w}\|_2^2] = 128\|\mathbf{w}\|_2^2$.

Attempt #4: Analysis

- Honest case: $\mathbb{E}[\|\mathbf{\Pi w}\|_2^2] = 128\|\mathbf{w}\|_2^2$.
- For $\mathbf{w} = (\vec{\mathbf{s}}_1, \ldots, \vec{\mathbf{s}}_r)$: if $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$, then $\sum_i \|\mathbf{s}_i\|_2^2 \leq \frac{128}{30}\beta^2$ except w.p. 2^{-128}: slack $\sqrt{128/30} \approx 2.07$.

Attempt #4: Analysis

- Honest case: $\mathbb{E}[\|\mathbf{\Pi w}\|_2^2] = 128\|\mathbf{w}\|_2^2$.
- For $\mathbf{w} = (\vec{\mathbf{s}}_1, \ldots, \vec{\mathbf{s}}_r)$: if $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$, then $\sum_i \|\mathbf{s}_i\|_2^2 \leq \frac{128}{30}\beta^2$ except w.p. 2^{-128}: slack $\sqrt{128/30} \approx 2.07$.
- The honest $\mathbf{p}$ passes only w.p. $\approx 1/2$: the prover asks for new matrices (affects only the running time).

Attempt #4: Analysis

- Honest case: $\mathbb{E}[\|\mathbf{\Pi w}\|_2^2] = 128\|\mathbf{w}\|_2^2$.
- For $\mathbf{w} = (\vec{\mathbf{s}}_1, \ldots, \vec{\mathbf{s}}_r)$: if $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$, then $\sum_i \|\mathbf{s}_i\|_2^2 \leq \frac{128}{30}\beta^2$ except w.p. 2^{-128}: slack $\sqrt{128/30} \approx 2.07$.
- The honest $\mathbf{p}$ passes only w.p. $\approx 1/2$: the prover asks for new matrices (affects only the running time).

Correctness of $\mathbf{p}$ for free. Let $\boldsymbol{\pi}_{i,j} \in \mathcal{R}_q^n$ have the j-th row of $\mathbf{\Pi}_i$ as coefficient vector. Then

$$\mathbf{ct}\Big(\sum_{i\in[r]} \langle \boldsymbol{\pi}_{i,j}^*, \mathbf{s}_i \rangle - p_j\Big) = 0$$

is a constant-coefficient constraint: add these 256 functions to $\mathcal{F}_{\mathsf{ct}}$.

Attempt #4: Analysis

- Honest case: $\mathbb{E}[\|\mathbf{\Pi w}\|_2^2] = 128\|\mathbf{w}\|_2^2$.
- For $\mathbf{w} = (\vec{\mathbf{s}}_1, \ldots, \vec{\mathbf{s}}_r)$: if $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$, then $\sum_i \|\mathbf{s}_i\|_2^2 \leq \frac{128}{30}\beta^2$ except w.p. 2^{-128}: slack $\sqrt{128/30} \approx 2.07$.
- The honest $\mathbf{p}$ passes only w.p. $\approx 1/2$: the prover asks for new matrices (affects only the running time).

Correctness of $\mathbf{p}$ for free. Let $\boldsymbol{\pi}_{i,j} \in \mathcal{R}_q^n$ have the j-th row of $\mathbf{\Pi}_i$ as coefficient vector. Then

$$\mathbf{ct}\Big(\sum_{i\in[r]} \langle \boldsymbol{\pi}_{i,j}^*, \mathbf{s}_i \rangle - p_j\Big) = 0$$

is a constant-coefficient constraint: add these 256 functions to $\mathcal{F}_{\mathsf{ct}}$.

Question #3

Let $\mathbf{w} \in \mathbb{Z}^m$ and $\boldsymbol{\pi} \leftarrow\!\$\ \mathcal{B}^m$. Show that $\mathbb{E}[\langle \boldsymbol{\pi}, \mathbf{w} \rangle^2] = \frac{1}{2}\|\mathbf{w}\|_2^2$, and deduce the expectation above.

The Base Protocol: Message Flow

Prover $\mathcal{P}$		Verifier $\mathcal{V}$
commit: $\mathbf{t}_i \leftarrow \mathbf{A}\mathbf{s}_i,\ i \in [r]$	$\xrightarrow{(\mathbf{t}_i)_i}$	
	$\xleftarrow{(\mathbf{\Pi}_i)_i}$	$\mathbf{\Pi}_i \leftarrow\$\ \mathcal{B}^{256\times nd}$

The Base Protocol: Message Flow

Prover $\mathcal{P}$		Verifier $\mathcal{V}$
commit: $\mathbf{t}_i \leftarrow \mathbf{A}\mathbf{s}_i,\ i \in [r]$	$\xrightarrow{(\mathbf{t}_i)_i}$	
	$\xleftarrow{(\mathbf{\Pi}_i)_i}$	$\mathbf{\Pi}_i \leftarrow\$\ \mathcal{B}^{256\times nd}$
	$\xrightarrow{\mathbf{p}}$	**assert** $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$
project: $\mathbf{p} \leftarrow \sum_i \mathbf{\Pi}_i \vec{\mathbf{s}}_i$		
	$\xleftarrow{(\boldsymbol{\psi}_k)_k}$	$\boldsymbol{\psi}_k \leftarrow\$\ \mathbb{Z}_q^{L+256}$

The Base Protocol: Message Flow

Prover $\mathcal{P}$		Verifier $\mathcal{V}$
commit: $\mathbf{t}_i \leftarrow \mathbf{A}\mathbf{s}_i,\ i \in [r]$	$\xrightarrow{(\mathbf{t}_i)_i}$	
	$\xleftarrow{(\mathbf{\Pi}_i)_i}$	$\mathbf{\Pi}_i \leftarrow\$\ \mathcal{B}^{256\times nd}$
	$\xrightarrow{\mathbf{p}}$	**assert** $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$
project: $\mathbf{p} \leftarrow \sum_i \mathbf{\Pi}_i \vec{\mathbf{s}}_i$		
	$\xleftarrow{(\boldsymbol{\psi}_k)_k}$	$\boldsymbol{\psi}_k \leftarrow\$\ \mathbb{Z}_q^{L+256}$
	$\xrightarrow{(v_k)_k}$	**assert** $\mathsf{ct}(v_k) = -\mathsf{ct}(b_k)$
aggregate: $v_k \leftarrow Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r)$		
	$\xleftarrow{\boldsymbol{\alpha}}$	$\boldsymbol{\alpha} \leftarrow\$\ \mathcal{R}_q^{K+K_{\mathsf{ct}}}$

The Base Protocol: Message Flow

Prover $\mathcal{P}$		Verifier $\mathcal{V}$
commit: $\mathbf{t}_i \leftarrow \mathbf{A}\mathbf{s}_i,\ i \in [r]$	$\xrightarrow{(\mathbf{t}_i)_i}$	
	$\xleftarrow{(\mathbf{\Pi}_i)_i}$	$\mathbf{\Pi}_i \leftarrow\$\ \mathcal{B}^{256\times nd}$
	$\xrightarrow{\mathbf{p}}$	**assert** $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$
project: $\mathbf{p} \leftarrow \sum_i \mathbf{\Pi}_i \vec{\mathbf{s}}_i$		
	$\xleftarrow{(\boldsymbol{\psi}_k)_k}$	$\boldsymbol{\psi}_k \leftarrow\$\ \mathbb{Z}_q^{L+256}$
	$\xrightarrow{(v_k)_k}$	**assert** $\mathsf{ct}(v_k) = -\mathsf{ct}(b_k)$
aggregate: $v_k \leftarrow Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r)$		
	$\xleftarrow{\boldsymbol{\alpha}}$	$\boldsymbol{\alpha} \leftarrow\$\ \mathcal{R}_q^{K+K_{\mathsf{ct}}}$
	$\xrightarrow{(g_{ij},h_{ij})_{i\leq j}}$	
aggregate into F; garbage g_{ij}, h_{ij}		
	$\xleftarrow{c_1,\ldots,c_r}$	$c_i \leftarrow\$\ C$

The Base Protocol: Message Flow

Prover $\mathcal{P}$		Verifier $\mathcal{V}$
commit: $\mathbf{t}_i \leftarrow \mathbf{A}\mathbf{s}_i,\ i \in [r]$	$\xrightarrow{(\mathbf{t}_i)_i}$	
	$\xleftarrow{(\boldsymbol{\Pi}_i)_i}$	$\boldsymbol{\Pi}_i \leftarrow\$\ \mathcal{B}^{256\times nd}$
	$\xrightarrow{\mathbf{p}}$	**assert** $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$
project: $\mathbf{p} \leftarrow \sum_i \boldsymbol{\Pi}_i \vec{\mathbf{s}}_i$		
	$\xleftarrow{(\boldsymbol{\psi}_k)_k}$	$\boldsymbol{\psi}_k \leftarrow\$\ \mathbb{Z}_q^{L+256}$
	$\xrightarrow{(v_k)_k}$	**assert** $\mathsf{ct}(v_k) = -\mathsf{ct}(b_k)$
aggregate: $v_k \leftarrow Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r)$		
	$\xleftarrow{\boldsymbol{\alpha}}$	$\boldsymbol{\alpha} \leftarrow\$\ \mathcal{R}_q^{K+K_{\mathsf{ct}}}$
	$\xrightarrow{(g_{ij},h_{ij})_{i\leq j}}$	
aggregate into F; garbage g_{ij}, h_{ij}		
	$\xleftarrow{c_1,\ldots,c_r}$	$c_i \leftarrow\$\ C$
	$\xrightarrow{\mathbf{z}}$	**assert** the checks of Attempt #2
amortize: $\mathbf{z} \leftarrow \sum_i c_i \mathbf{s}_i$		

The Base Protocol: Message Flow

Prover $\mathcal{P}$		Verifier $\mathcal{V}$
commit: $\mathbf{t}_i \leftarrow \mathbf{A}\mathbf{s}_i$, $i \in [r]$	$\xrightarrow{(\mathbf{t}_i)_i}$	
	$\xleftarrow{(\mathbf{\Pi}_i)_i}$	$\mathbf{\Pi}_i \leftarrow\$ \mathcal{B}^{256\times nd}$
	$\xrightarrow{\mathbf{p}}$	**assert** $\|\mathbf{p}\|_2 \leq \sqrt{128}\beta$
project: $\mathbf{p} \leftarrow \sum_i \mathbf{\Pi}_i \vec{\mathbf{s}}_i$		
	$\xleftarrow{(\boldsymbol{\psi}_k)_k}$	$\boldsymbol{\psi}_k \leftarrow\$ \mathbb{Z}_q^{L+256}$
	$\xrightarrow{(v_k)_k}$	**assert** $\mathrm{ct}(v_k) = -\mathrm{ct}(b_k)$
aggregate: $v_k \leftarrow Q_k(\mathbf{s}_1, \ldots, \mathbf{s}_r)$		
	$\xleftarrow{\boldsymbol{\alpha}}$	$\boldsymbol{\alpha} \leftarrow\$ \mathcal{R}_q^{K+K_{\mathrm{ct}}}$
	$\xrightarrow{(g_{ij},h_{ij})_{i\leq j}}$	
aggregate into F; garbage g_{ij}, h_{ij}		
	$\xleftarrow{c_1,\ldots,c_r}$	$c_i \leftarrow\$ C$
	$\xrightarrow{\mathbf{z}}$	**assert** the checks of Attempt #2
amortize: $\mathbf{z} \leftarrow \sum_i c_i \mathbf{s}_i$		

Order matters: the projection is aggregated, so it comes before $\boldsymbol{\psi}_k$; h_{ij} depend on the aggregated $\boldsymbol{\varphi}_i$, so they come after $\boldsymbol{\alpha}$.

Recursion

The Base Protocol Is Not Succinct

Its proof contains $r\kappa$ commitments, $r^2 + r$ garbage polynomials, and $\mathbf{z}$ of rank n.

The Base Protocol Is Not Succinct

Its proof contains $r\kappa$ commitments, $r^2 + r$ garbage polynomials, and $\mathbf{z}$ of rank n.

Observation

All checks on the last message are **dot product constraints**:

- $\mathbf{Az} = \sum_i c_i \mathbf{t}_i$ is linear in $\mathbf{z}$ and $\mathbf{t}_i$;
- $\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j} g_{ij} c_i c_j$ is quadratic in $\mathbf{z}$ and linear in g_{ij};
- the remaining two equations are linear in $\mathbf{z}$, g_{ij}, h_{ij}.

All coefficients (c_i, $\mathbf{A}$, a_{ij}, $\boldsymbol{\varphi}_i$, a_0) are known to the verifier.

The Base Protocol Is Not Succinct

Its proof contains $r\kappa$ commitments, $r^2 + r$ garbage polynomials, and $\mathbf{z}$ of rank n.

Observation

All checks on the last message are **dot product constraints**:

- $\mathbf{Az} = \sum_i c_i \mathbf{t}_i$ is linear in $\mathbf{z}$ and $\mathbf{t}_i$;
- $\langle \mathbf{z}, \mathbf{z} \rangle = \sum_{i,j} g_{ij} c_i c_j$ is quadratic in $\mathbf{z}$ and linear in g_{ij};
- the remaining two equations are linear in $\mathbf{z}$, g_{ij}, h_{ij}.

All coefficients (c_i, $\mathbf{A}$, a_{ij}, $\boldsymbol{\varphi}_i$, a_0) are known to the verifier.

The last message is a witness for a *new* instance of $\mathcal{R}_{\text{dot}}$: prove knowledge of it by running the protocol again!

Warm-up: Proof Compression

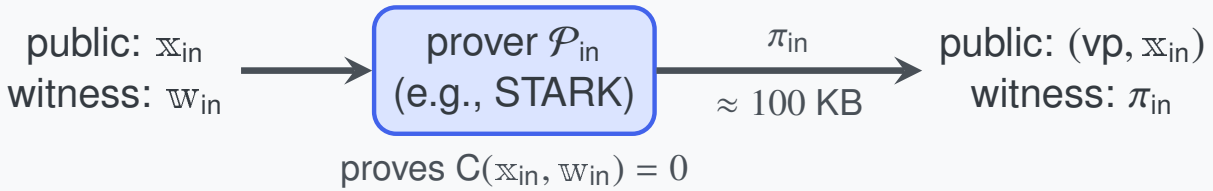
Fast prover with large proofs (STARK, ≈ 100 KB) + slow prover with tiny proofs (Groth16). Prove "I know an accepting inner proof":

$$\mathcal{R}_{\mathsf{out}} := \{(\mathbb{x}_{\mathsf{out}} := (\mathsf{vp}, \mathbb{x}_{\mathsf{in}}), \mathbb{w}_{\mathsf{out}} := \pi_{\mathsf{in}}) : \mathcal{V}_{\mathsf{in}}(\mathsf{vp}, \mathbb{x}_{\mathsf{in}}, \pi_{\mathsf{in}}) = 1\}.$$

Warm-up: Proof Compression

Fast prover with large proofs (STARK, ≈ 100 KB) + slow prover with tiny proofs (Groth16). Prove "I know an accepting inner proof":

$$\mathcal{R}_{\text{out}} := \{(\mathbb{x}_{\text{out}} := (\text{vp}, \mathbb{x}_{\text{in}}), \mathbb{w}_{\text{out}} := \pi_{\text{in}}) : \mathcal{V}_{\text{in}}(\text{vp}, \mathbb{x}_{\text{in}}, \pi_{\text{in}}) = 1\}.$$

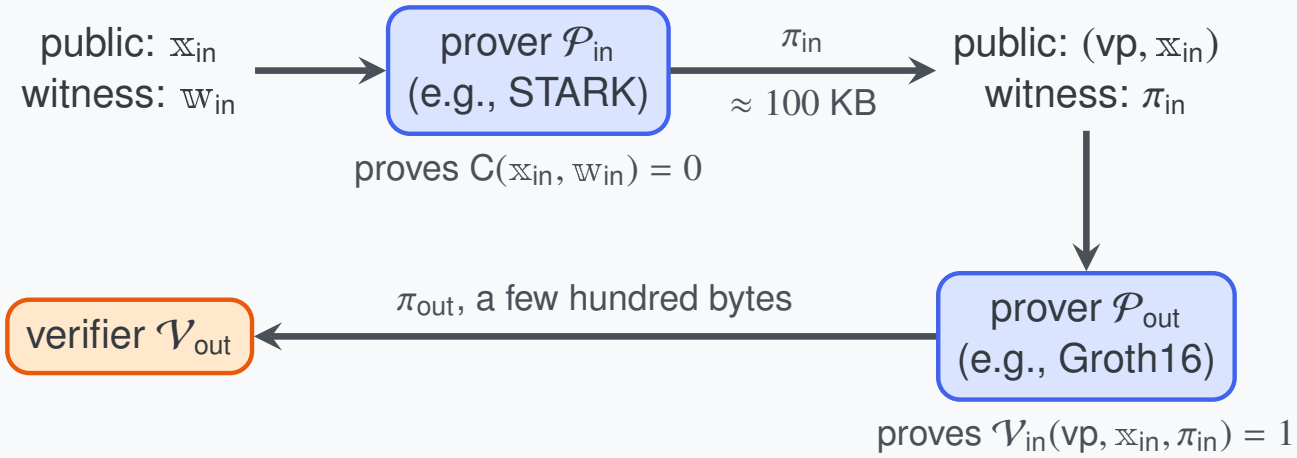


The slow prover handles only the (small) verification circuit of $\mathcal{V}_{\text{in}}$. Used in practice to post STARK proofs on-chain.

Warm-up: Proof Compression

Fast prover with large proofs (STARK, ≈ 100 KB) + slow prover with tiny proofs (Groth16). Prove "I know an accepting inner proof":

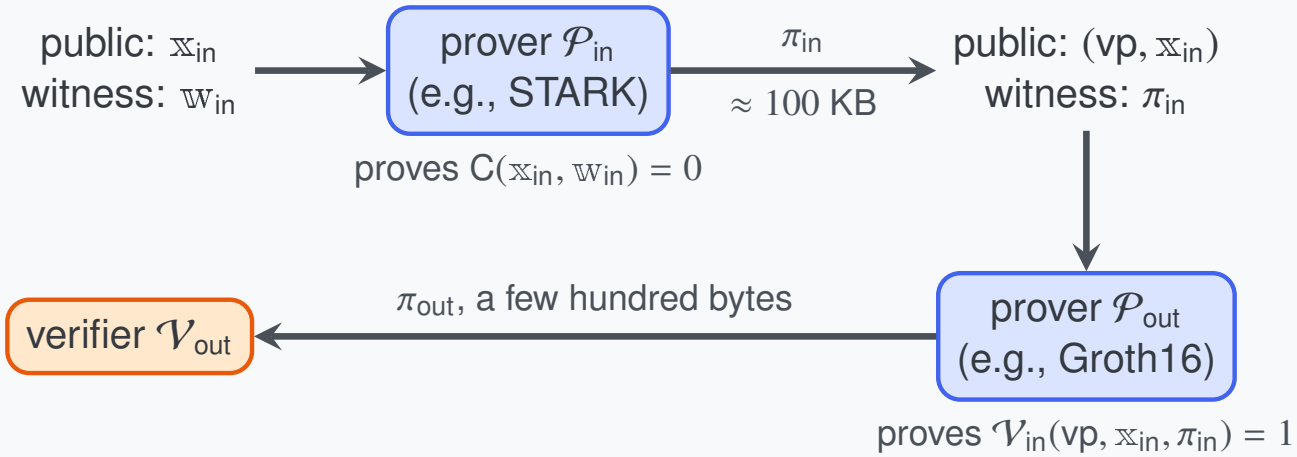
$$\mathcal{R}_{\text{out}} := \{(\mathbb{x}_{\text{out}} := (\text{vp}, \mathbb{x}_{\text{in}}), \mathbb{w}_{\text{out}} := \pi_{\text{in}}) : \mathcal{V}_{\text{in}}(\text{vp}, \mathbb{x}_{\text{in}}, \pi_{\text{in}}) = 1\}.$$



Warm-up: Proof Compression

Fast prover with large proofs (STARK, ≈ 100 KB) + slow prover with tiny proofs (Groth16). Prove "I know an accepting inner proof":

$$\mathcal{R}_{\text{out}} := \{(\mathbb{x}_{\text{out}} := (\text{vp}, \mathbb{x}_{\text{in}}), \mathbb{w}_{\text{out}} := \pi_{\text{in}}) : \mathcal{V}_{\text{in}}(\text{vp}, \mathbb{x}_{\text{in}}, \pi_{\text{in}}) = 1\}.$$



The slow prover handles only the (small) verification circuit of $\mathcal{V}_{\text{in}}$. Used in practice to post STARK proofs on-chain.

Knowledge Soundness of the Composition

Proposition

Let κ_{in} and κ_{out} be the knowledge errors of inner and outer systems. The resultant proving system has a knowledge error of $\kappa_{\text{in}} + \kappa_{\text{out}}$.

Knowledge Soundness of the Composition

Proposition

Let κ_{in} and κ_{out} be the knowledge errors of inner and outer systems. The resultant proving system has a knowledge error of $\kappa_{\text{in}} + \kappa_{\text{out}}$.

Proof idea. Let $\mathcal{A}$ convince $\mathcal{V}_{\text{out}}$ with probability ϵ.

1: $\mathcal{E}_{\text{out}}$ obtains an accepting π_{in} w.p. $\geq \epsilon - \kappa_{\text{out}}$;

Knowledge Soundness of the Composition

Proposition

Let κ_{in} and κ_{out} be the knowledge errors of inner and outer systems. The resultant proving system has a knowledge error of $\kappa_{\text{in}} + \kappa_{\text{out}}$.

Proof idea. Let $\mathcal{A}$ convince $\mathcal{V}_{\text{out}}$ with probability ϵ.

1: $\mathcal{E}_{\text{out}}$ obtains an accepting π_{in} w.p. $\geq \epsilon - \kappa_{\text{out}}$;

2: so $\mathcal{E}_{\text{out}}$ (run on $\mathcal{A}$) is a successful inner prover;

Knowledge Soundness of the Composition

Proposition

Let κ_{in} and κ_{out} be the knowledge errors of inner and outer systems. The resultant proving system has a knowledge error of $\kappa_{\text{in}} + \kappa_{\text{out}}$.

Proof idea. Let $\mathcal{A}$ convince $\mathcal{V}_{\text{out}}$ with probability ϵ.

1: $\mathcal{E}_{\text{out}}$ obtains an accepting π_{in} w.p. $\geq \epsilon - \kappa_{\text{out}}$;
2: so $\mathcal{E}_{\text{out}}$ (run on $\mathcal{A}$) is a successful inner prover;
3: $\mathcal{E}_{\text{in}}$, run on $\mathcal{E}_{\text{out}}$, obtains $\mathbb{w}$ w.p. $\geq \epsilon - (\kappa_{\text{out}} + \kappa_{\text{in}})$. ■

Knowledge Soundness of the Composition

Proposition

Let κ_{in} and κ_{out} be the knowledge errors of inner and outer systems. The resultant proving system has a knowledge error of $\kappa_{\text{in}} + \kappa_{\text{out}}$.

Proof idea. Let $\mathcal{A}$ convince $\mathcal{V}_{\text{out}}$ with probability ϵ.

1: $\mathcal{E}_{\text{out}}$ obtains an accepting π_{in} w.p. $\geq \epsilon - \kappa_{\text{out}}$;
2: so $\mathcal{E}_{\text{out}}$ (run on $\mathcal{A}$) is a successful inner prover;
3: $\mathcal{E}_{\text{in}}$, run on $\mathcal{E}_{\text{out}}$, obtains $\mathbb{w}$ w.p. $\geq \epsilon - (\kappa_{\text{out}} + \kappa_{\text{in}})$. ■

Caveats

1. Extractors are *nested*: k levels may cost 2^k times longer, so only a few levels (at most logarithmic in λ).
2. With Fiat-Shamir inside, the circuit contains a concrete hash function: a heuristic assumption.

Reductions of Knowledge

An interactive protocol may not decide whether to accept, but *reduce* a claim about one relation to a claim about another. A relation $\mathcal{R}$ consists of triples $(\mathsf{pp}, \mathbb{x}, \mathbb{w})$.

Definition (Reduction of Knowledge)

A **reduction of knowledge** Π from $\mathcal{R}_1$ to $\mathcal{R}_2$ consists of

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$;
- $\langle \mathcal{P}(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1), \mathcal{V}(\mathsf{pp}, \mathbb{x}_1) \rangle \to (\mathbb{x}_2, \mathbb{w}_2)$: the verifier outputs a new statement $\mathbb{x}_2$ (or $\perp$), the prover a new witness $\mathbb{w}_2$.

Reductions of Knowledge

An interactive protocol may not decide whether to accept, but *reduce* a claim about one relation to a claim about another. A relation $\mathcal{R}$ consists of triples $(\mathsf{pp}, \mathbb{x}, \mathbb{w})$.

Definition (Reduction of Knowledge)

A **reduction of knowledge** Π from $\mathcal{R}_1$ to $\mathcal{R}_2$ consists of

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$;
- $\langle \mathcal{P}(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1), \mathcal{V}(\mathsf{pp}, \mathbb{x}_1) \rangle \to (\mathbb{x}_2, \mathbb{w}_2)$: the verifier outputs a new statement $\mathbb{x}_2$ (or $\perp$), the prover a new witness $\mathbb{w}_2$.

- **Completeness**: honest $(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1) \in \mathcal{R}_1$ gives $(\mathsf{pp}, \mathbb{x}_2, \mathbb{w}_2) \in \mathcal{R}_2$.

Reductions of Knowledge

An interactive protocol may not decide whether to accept, but *reduce* a claim about one relation to a claim about another. A relation $\mathcal{R}$ consists of triples $(\mathsf{pp}, \mathbb{x}, \mathbb{w})$.

Definition (Reduction of Knowledge)

A **reduction of knowledge** Π from $\mathcal{R}_1$ to $\mathcal{R}_2$ consists of

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$;
- $\langle \mathcal{P}(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1), \mathcal{V}(\mathsf{pp}, \mathbb{x}_1) \rangle \to (\mathbb{x}_2, \mathbb{w}_2)$: the verifier outputs a new statement $\mathbb{x}_2$ (or $\perp$), the prover a new witness $\mathbb{w}_2$.

- **Completeness**: honest $(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1) \in \mathcal{R}_1$ gives $(\mathsf{pp}, \mathbb{x}_2, \mathbb{w}_2) \in \mathcal{R}_2$.
- **Knowledge soundness** (error κ): if $\mathcal{P}^*$ ends in $\mathcal{R}_2$ w.p. $\epsilon > \kappa$, an expected poly-time extractor outputs $\mathbb{w}_1$ with $(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1) \in \mathcal{R}_1$ w.p. $\geq \epsilon - \kappa$.

Reductions of Knowledge

An interactive protocol may not decide whether to accept, but *reduce* a claim about one relation to a claim about another. A relation $\mathcal{R}$ consists of triples $(\mathsf{pp}, \mathbb{x}, \mathbb{w})$.

Definition (Reduction of Knowledge)

A **reduction of knowledge** Π from $\mathcal{R}_1$ to $\mathcal{R}_2$ consists of

- $\mathsf{Setup}(1^\lambda) \to \mathsf{pp}$;
- $\langle \mathcal{P}(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1), \mathcal{V}(\mathsf{pp}, \mathbb{x}_1) \rangle \to (\mathbb{x}_2, \mathbb{w}_2)$: the verifier outputs a new statement $\mathbb{x}_2$ (or $\perp$), the prover a new witness $\mathbb{w}_2$.

- **Completeness**: honest $(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1) \in \mathcal{R}_1$ gives $(\mathsf{pp}, \mathbb{x}_2, \mathbb{w}_2) \in \mathcal{R}_2$.
- **Knowledge soundness** (error κ): if $\mathcal{P}^*$ ends in $\mathcal{R}_2$ w.p. $\epsilon > \kappa$, an expected poly-time extractor outputs $\mathbb{w}_1$ with $(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1) \in \mathcal{R}_1$ w.p. $\geq \epsilon - \kappa$.
- **Public reducibility**: $\mathbb{x}_2 = \mathsf{f}(\mathsf{pp}, \mathbb{x}_1, \mathsf{tr})$ for a deterministic poly-time f.

Knowledge Soundness, Formally

There exists an expected polynomial-time $\mathcal{E}$ such that for every expected polynomial-time $(\mathcal{A}, \mathcal{P}^*)$, if

$$\Pr\left[(\mathsf{pp}, \mathbb{x}_2, \mathbb{w}_2) \in \mathcal{R}_2 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda), \\ (\mathbb{x}_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}), \\ (\mathbb{x}_2, \mathbb{w}_2) \leftarrow \langle \mathcal{P}^*(\mathsf{st}), \mathcal{V}(\mathsf{pp}, \mathbb{x}_1) \rangle \end{array}\right] = \epsilon > \kappa,$$

then $\mathcal{E}^{\mathcal{A}, \mathcal{P}^*}(\mathsf{pp}, \mathbb{x}_1)$ outputs $\mathbb{w}_1$ with $(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1) \in \mathcal{R}_1$ with probability at least $\epsilon - \kappa$.

Knowledge Soundness, Formally

There exists an expected polynomial-time $\mathcal{E}$ such that for every expected polynomial-time $(\mathcal{A}, \mathcal{P}^*)$, if

$$\Pr\left[(\mathsf{pp}, \mathbb{x}_2, \mathbb{w}_2) \in \mathcal{R}_2 \;\middle|\; \begin{array}{l} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda), \\ (\mathbb{x}_1, \mathsf{st}) \leftarrow \mathcal{A}(\mathsf{pp}), \\ (\mathbb{x}_2, \mathbb{w}_2) \leftarrow \langle \mathcal{P}^*(\mathsf{st}), \mathcal{V}(\mathsf{pp}, \mathbb{x}_1) \rangle \end{array}\right] = \epsilon > \kappa,$$

then $\mathcal{E}^{\mathcal{A}, \mathcal{P}^*}(\mathsf{pp}, \mathbb{x}_1)$ outputs $\mathbb{w}_1$ with $(\mathsf{pp}, \mathbb{x}_1, \mathbb{w}_1) \in \mathcal{R}_1$ with probability at least $\epsilon - \kappa$.

In words

Whoever can produce a witness for the new statement $\mathbb{x}_2$ must have known a witness for the original statement $\mathbb{x}_1$.

Examples and Composition

Example

1. An argument of knowledge for $\mathcal{R}$ = a reduction to $\mathcal{R}_\top := \{(\mathsf{pp}, \top, \bot)\}$ ($\top$ means "accept").
2. Proof compression: $\mathcal{R}_{\mathsf{in}} \to \mathcal{R}_{\mathsf{out}} \to \mathcal{R}_\top$.
3. Π_{send}: send $\mathbb{w}$, output $\top$ iff $(\mathsf{pp}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$. Knowledge error 0, not succinct.

Examples and Composition

Example

1. An argument of knowledge for $\mathcal{R}$ = a reduction to $\mathcal{R}_\top := \{(\mathsf{pp}, \top, \bot)\}$ ($\top$ means "accept").
2. Proof compression: $\mathcal{R}_{\mathsf{in}} \to \mathcal{R}_{\mathsf{out}} \to \mathcal{R}_\top$.
3. Π_{send}: send $\mathbb{w}$, output $\top$ iff $(\mathsf{pp}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$. Knowledge error 0, not succinct.

Theorem (Composition)

Let Π be a reduction of knowledge from $\mathcal{R}$ to $\mathcal{R}'$ with knowledge error κ.

(i) **Sequential.** If Π' reduces $\mathcal{R}'$ to $\mathcal{R}''$ with error κ', then $\Pi' \circ \Pi$ reduces $\mathcal{R}$ to $\mathcal{R}''$ with error $\kappa + \kappa'$.

(ii) **Parallel.** If $\widetilde{\Pi}$ reduces $\widetilde{\mathcal{R}}$ to $\widetilde{\mathcal{R}}'$, then $\Pi \times \widetilde{\Pi}$ reduces $\mathcal{R} \times \widetilde{\mathcal{R}}$ to $\mathcal{R}' \times \widetilde{\mathcal{R}}'$.

Examples and Composition

Example

1. An argument of knowledge for $\mathcal{R}$ = a reduction to $\mathcal{R}_\top := \{(\mathsf{pp}, \top, \bot)\}$ ($\top$ means "accept").
2. Proof compression: $\mathcal{R}_{\mathsf{in}} \to \mathcal{R}_{\mathsf{out}} \to \mathcal{R}_\top$.
3. Π_{send}: send $\mathbb{w}$, output $\top$ iff $(\mathsf{pp}, \mathbb{x}, \mathbb{w}) \in \mathcal{R}$. Knowledge error 0, not succinct.

Theorem (Composition)

Let Π be a reduction of knowledge from $\mathcal{R}$ to $\mathcal{R}'$ with knowledge error κ.

(i) **Sequential.** If Π' reduces $\mathcal{R}'$ to $\mathcal{R}''$ with error κ', then $\Pi' \circ \Pi$ reduces $\mathcal{R}$ to $\mathcal{R}''$ with error $\kappa + \kappa'$.

(ii) **Parallel.** If $\widetilde{\Pi}$ reduces $\widetilde{\mathcal{R}}$ to $\widetilde{\mathcal{R}}'$, then $\Pi \times \widetilde{\Pi}$ reduces $\mathcal{R} \times \widetilde{\mathcal{R}}$ to $\mathcal{R}' \times \widetilde{\mathcal{R}}'$.

Proof of *(i)*: exactly the nested extraction of the warm-up.

LaBRADOR as a Chain of Reductions

💬 *Relaxed relations*

Lattice reductions are complete for one relation but knowledge sound only for a *relaxed* one: e.g., complete for norm $\leq \beta$, extraction only guarantees $\leq \sqrt{128/30}\beta$. LaBRADOR chooses the Module-SIS parameters of each iteration with the slack of the next ones in mind.

LaBRADOR as a Chain of Reductions

Relaxed relations

Lattice reductions are complete for one relation but knowledge sound only for a *relaxed* one: e.g., complete for norm $\leq \beta$, extraction only guarantees $\leq \sqrt{128/30}\beta$. LaBRADOR chooses the Module-SIS parameters of each iteration with the slack of the next ones in mind.

Each iteration is $\Pi_{\mathsf{main}} : \mathcal{R}_{\mathsf{dot}} \to \mathcal{R}'_{\mathsf{dot}}$ (new parameters), and $\Pi_{\mathsf{R1CS}} : \mathcal{R}_{\mathsf{R1CS}} \to \mathcal{R}_{\mathsf{dot}}$. The complete protocol is

$$\Pi_{\mathsf{LaBRADOR}} := \Pi_{\mathsf{send}} \circ \underbrace{\Pi_{\mathsf{main}} \circ \cdots \circ \Pi_{\mathsf{main}}}_{\ell \text{ times}} \circ \Pi_{\mathsf{R1CS}},$$

with knowledge error $\kappa_{\mathsf{R1CS}} + \ell \cdot \kappa_{\mathsf{main}}$.

LaBRADOR as a Chain of Reductions

Relaxed relations

Lattice reductions are complete for one relation but knowledge sound only for a *relaxed* one: e.g., complete for norm $\leq \beta$, extraction only guarantees $\leq \sqrt{128/30}\beta$. LaBRADOR chooses the Module-SIS parameters of each iteration with the slack of the next ones in mind.

Each iteration is $\Pi_{\mathsf{main}} : \mathcal{R}_{\mathsf{dot}} \to \mathcal{R}'_{\mathsf{dot}}$ (new parameters), and $\Pi_{\mathsf{R1CS}} : \mathcal{R}_{\mathsf{R1CS}} \to \mathcal{R}_{\mathsf{dot}}$. The complete protocol is

$$\Pi_{\mathsf{LaBRADOR}} := \Pi_{\mathsf{send}} \circ \underbrace{\Pi_{\mathsf{main}} \circ \cdots \circ \Pi_{\mathsf{main}}}_{\ell \text{ times}} \circ \Pi_{\mathsf{R1CS}},$$

with knowledge error $\kappa_{\mathsf{R1CS}} + \ell \cdot \kappa_{\mathsf{main}}$.

- Same idea as folding schemes (Nova, LatticeFold).
- Both caveats disappear: $O(\log \log N)$ iterations, and Fiat-Shamir is applied once to the whole public-coin protocol.

Three Issues

To turn the base protocol into a reduction from $\mathcal{R}_{\text{dot}}$ to itself:

- *Issue #1:* the commitments $\mathbf{t}_i$ and the garbage are sent in the clear, and their size grows with r;

Three Issues

To turn the base protocol into a reduction from $\mathcal{R}_{\mathsf{dot}}$ to itself:

- *Issue #1:* the commitments $\mathbf{t}_i$ and the garbage are sent in the clear, and their size grows with r;
- *Issue #2:* the coefficients of $\mathbf{z}$ are larger by $\approx \sqrt{r\tau}$: they would blow up after a few iterations;

Three Issues

To turn the base protocol into a reduction from $\mathcal{R}_{\text{dot}}$ to itself:

- *Issue #1:* the commitments $\mathbf{t}_i$ and the garbage are sent in the clear, and their size grows with r;
- *Issue #2:* the coefficients of $\mathbf{z}$ are larger by $\approx \sqrt{r\tau}$: they would blow up after a few iterations;
- *Issue #3:* the new witness is one long $\mathbf{z}$ and several short vectors, while the protocol prefers many vectors of the same rank.

Three Issues

To turn the base protocol into a reduction from $\mathcal{R}_{\text{dot}}$ to itself:

- *Issue #1:* the commitments $\mathbf{t}_i$ and the garbage are sent in the clear, and their size grows with r;
- *Issue #2:* the coefficients of $\mathbf{z}$ are larger by $\approx \sqrt{r\tau}$: they would blow up after a few iterations;
- *Issue #3:* the new witness is one long $\mathbf{z}$ and several short vectors, while the protocol prefers many vectors of the same rank.

Outer commitments (Issue #1)

Move commitments and garbage into the new witness: the verifier need not *see* them, they just must be fixed before the challenges. Send a single Ajtai commitment to all **inner commitments** $\mathbf{t}_i$: an **outer commitment**.

Outer Commitments

Ajtai needs short messages, but $\mathbf{t}_i$ has arbitrary coefficients. Decompose in a small base δ:

$$\mathbf{t}_i = \sum_{k \in [t]} \mathbf{t}_{i,k} \delta^k, \quad \|\mathbf{t}_{i,k}\|_\infty \leq \frac{1}{2}\delta, \quad t = \lceil \log q / \log \delta \rceil;$$

similarly h_{ij} in base δ and the (shorter) g_{ij} in base δ'. Collect the parts into $\mathbf{t}$, $\mathbf{g}$, $\mathbf{h}$.

Outer Commitments

Ajtai needs short messages, but $\mathbf{t}_i$ has arbitrary coefficients. Decompose in a small base δ:

$$\mathbf{t}_i = \sum_{k \in [t]} \mathbf{t}_{i,k} \delta^k, \quad \|\mathbf{t}_{i,k}\|_\infty \leq \frac{1}{2}\delta, \quad t = \lceil \log q / \log \delta \rceil;$$

similarly h_{ij} in base δ and the (shorter) g_{ij} in base δ'. Collect the parts into $\mathbf{t}$, $\mathbf{g}$, $\mathbf{h}$.

The prover sends two outer commitments

$$\mathbf{u}' := \mathbf{B}\mathbf{t} + \mathbf{C}\mathbf{g} \in \mathcal{R}_q^{\kappa'}, \qquad \mathbf{u}'' := \mathbf{D}\mathbf{h} \in \mathcal{R}_q^{\kappa''}.$$

Outer Commitments

Ajtai needs short messages, but $\mathbf{t}_i$ has arbitrary coefficients. Decompose in a small base δ:

$$\mathbf{t}_i = \sum_{k \in [t]} \mathbf{t}_{i,k} \delta^k, \quad \|\mathbf{t}_{i,k}\|_\infty \leq \frac{1}{2}\delta, \quad t = \lceil \log q / \log \delta \rceil;$$

similarly h_{ij} in base δ and the (shorter) g_{ij} in base δ'. Collect the parts into $\mathbf{t}$, $\mathbf{g}$, $\mathbf{h}$.

The prover sends two outer commitments

$$\mathbf{u}' := \mathbf{B}\mathbf{t} + \mathbf{C}\mathbf{g} \in \mathcal{R}_q^{\kappa'}, \qquad \mathbf{u}'' := \mathbf{D}\mathbf{h} \in \mathcal{R}_q^{\kappa''}.$$

- Two of them, as they are sent at different times: $\mathbf{t}_i$ and g_{ij} depend only on the witness (first message); h_{ij} depend on $\boldsymbol{\varphi}_i$ (after aggregation).
- Both equations are linear: $\kappa' + \kappa''$ additional constraints of the next instance.

Decomposing and Reshaping

Issue #2. Decompose $\mathbf{z} = \mathbf{z}_{\text{low}} + \delta'' \mathbf{z}_{\text{high}}$ with $\|\mathbf{z}_{\text{low}}\|_\infty \leq \frac{1}{2}\delta''$; the quadratic check becomes

$$\langle \mathbf{z}_{\text{low}}, \mathbf{z}_{\text{low}} \rangle + 2\delta'' \langle \mathbf{z}_{\text{low}}, \mathbf{z}_{\text{high}} \rangle + \delta''^2 \langle \mathbf{z}_{\text{high}}, \mathbf{z}_{\text{high}} \rangle = \sum_{i,j} g_{ij} c_i c_j,$$

still a dot product constraint. Choose $\delta \approx \delta' \approx \delta''$: all parts about equally wide.

Decomposing and Reshaping

Issue #2. Decompose $\mathbf{z} = \mathbf{z}_{\mathsf{low}} + \delta'' \mathbf{z}_{\mathsf{high}}$ with $\|\mathbf{z}_{\mathsf{low}}\|_\infty \leq \frac{1}{2}\delta''$; the quadratic check becomes

$$\langle \mathbf{z}_{\mathsf{low}}, \mathbf{z}_{\mathsf{low}} \rangle + 2\delta'' \langle \mathbf{z}_{\mathsf{low}}, \mathbf{z}_{\mathsf{high}} \rangle + \delta''^2 \langle \mathbf{z}_{\mathsf{high}}, \mathbf{z}_{\mathsf{high}} \rangle = \sum_{i,j} g_{ij} c_i c_j,$$

still a dot product constraint. Choose $\delta \approx \delta' \approx \delta''$: all parts about equally wide.

Issue #3. Let $\mathbf{o} := \mathbf{t} \,\|\, \mathbf{g} \,\|\, \mathbf{h} \in \mathcal{R}_q^m$. Split $\mathbf{z}_{\mathsf{low}}, \mathbf{z}_{\mathsf{high}}$ into ν pieces and $\mathbf{o}$ into μ pieces, pad to rank $n' := \max\{\lceil n/\nu \rceil, \lceil m/\mu \rceil\}$: a witness of multiplicity $r' = 2\nu + \mu$ ($n/\nu \approx m/\mu$).

Decomposing and Reshaping

Issue #2. Decompose $\mathbf{z} = \mathbf{z}_{\mathsf{low}} + \delta'' \mathbf{z}_{\mathsf{high}}$ with $\|\mathbf{z}_{\mathsf{low}}\|_\infty \leq \frac{1}{2}\delta''$; the quadratic check becomes

$$\langle \mathbf{z}_{\mathsf{low}}, \mathbf{z}_{\mathsf{low}} \rangle + 2\delta'' \langle \mathbf{z}_{\mathsf{low}}, \mathbf{z}_{\mathsf{high}} \rangle + \delta''^2 \langle \mathbf{z}_{\mathsf{high}}, \mathbf{z}_{\mathsf{high}} \rangle = \sum_{i,j} g_{ij} c_i c_j,$$

still a dot product constraint. Choose $\delta \approx \delta' \approx \delta''$: all parts about equally wide.

Issue #3. Let $\mathbf{o} := \mathbf{t} \,\|\, \mathbf{g} \,\|\, \mathbf{h} \in \mathcal{R}_q^m$. Split $\mathbf{z}_{\mathsf{low}}, \mathbf{z}_{\mathsf{high}}$ into ν pieces and $\mathbf{o}$ into μ pieces, pad to rank $n' := \max\{\lceil n/\nu \rceil, \lceil m/\mu \rceil\}$: a witness of multiplicity $r' = 2\nu + \mu$ ($n/\nu \approx m/\mu$).

The norm checks combine into one:

$$\sum_{i \in [r']} \|\mathbf{s}'_i\|_2^2 = \|\mathbf{z}_{\mathsf{low}}\|_2^2 + \|\mathbf{z}_{\mathsf{high}}\|_2^2 + \|\mathbf{o}\|_2^2 \leq \beta'^2.$$

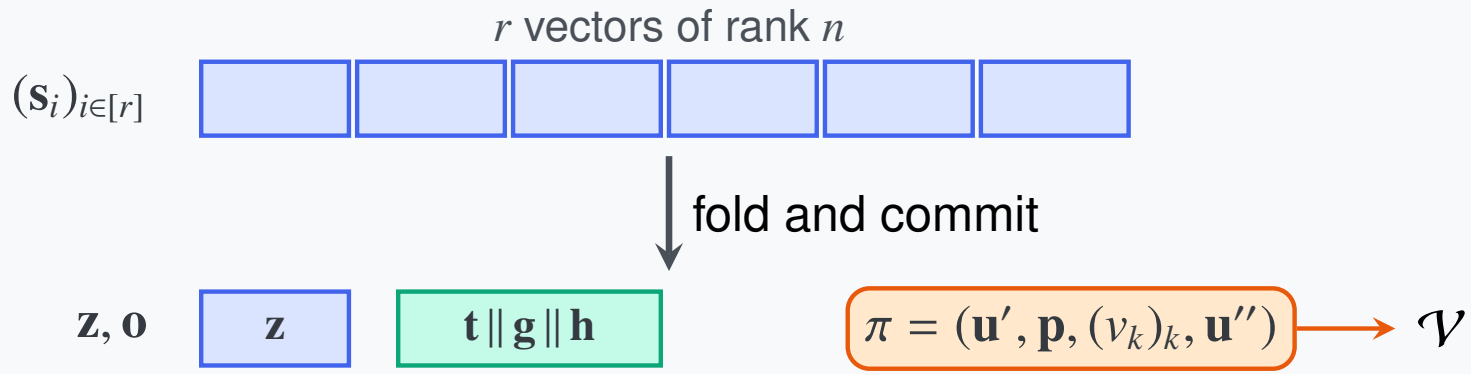
New instance: $\kappa + \kappa' + \kappa'' + 3$ constraints in $\mathcal{F}$, empty $\mathcal{F}_{\mathsf{ct}}$, bound β'.

One Iteration of the Main Protocol



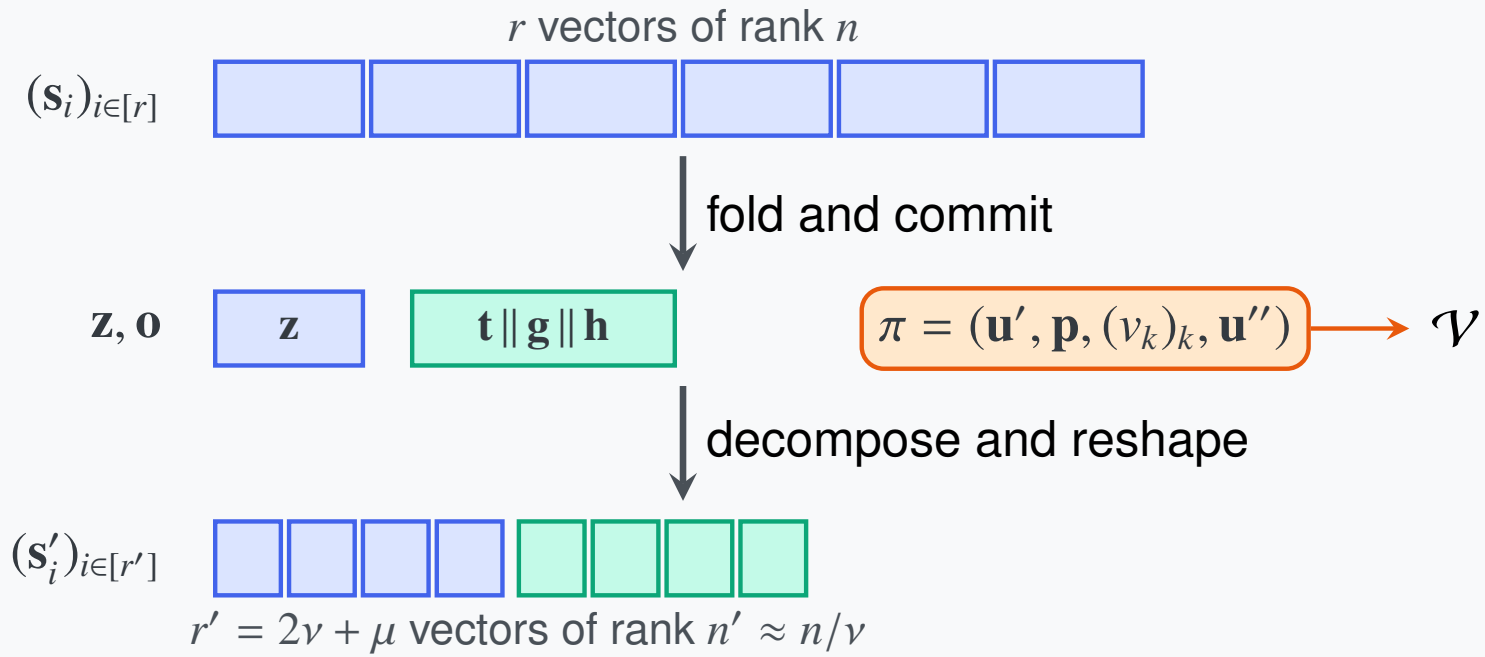
Instead of sending its last message, the prover outputs the new witness $(\mathbf{s}'_i)_{i\in[r']}$; the verifier outputs the new statement $(\mathcal{F}', \varnothing, \beta')$: $\mathbf{Az} = \sum_i c_i \mathbf{t}_i$, $\mathbf{u}' = \mathbf{Bt} + \mathbf{Cg}$, $\mathbf{u}'' = \mathbf{Dh}$, and the three garbage equations, rewritten in terms of the new witness.

One Iteration of the Main Protocol

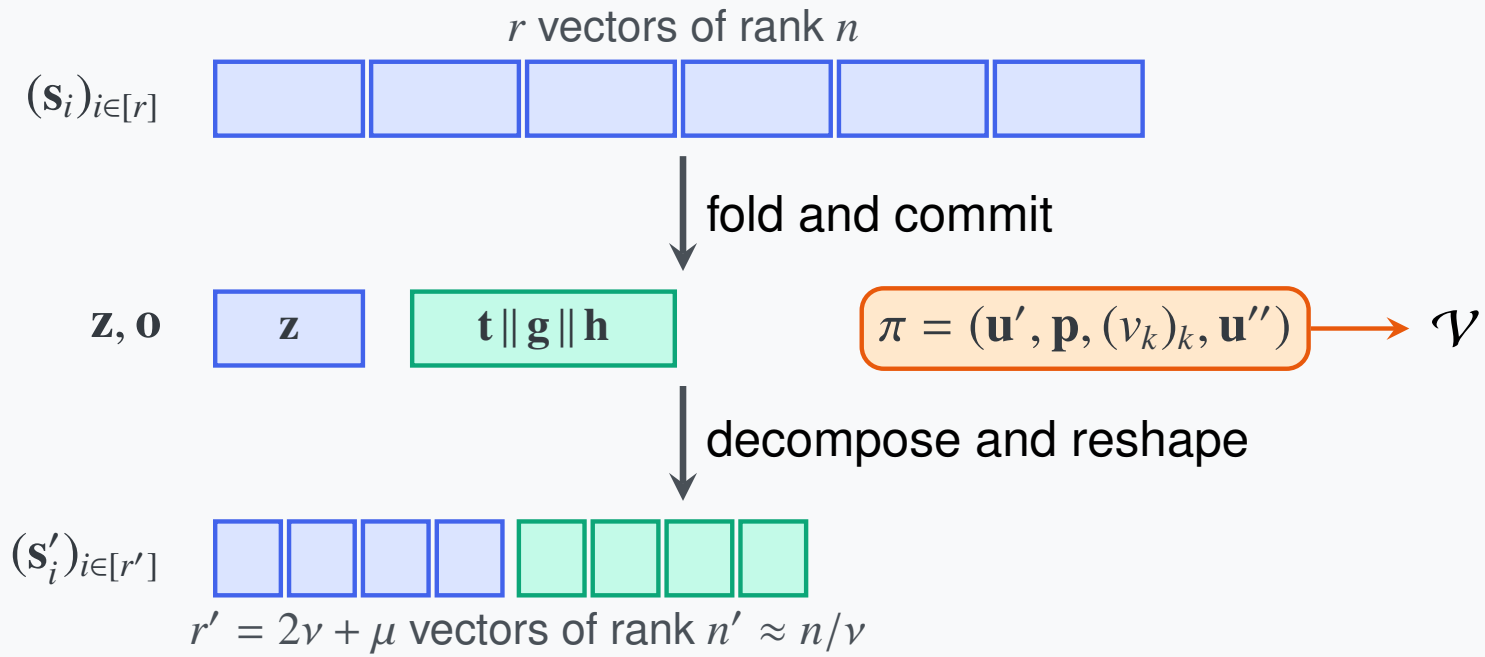


Instead of sending its last message, the prover outputs the new witness $(\mathbf{s}'_i)_{i\in[r']}$; the verifier outputs the new statement $(\mathcal{F}', \varnothing, \beta')$: $\mathbf{Az} = \sum_i c_i \mathbf{t}_i$, $\mathbf{u}' = \mathbf{Bt} + \mathbf{Cg}$, $\mathbf{u}'' = \mathbf{Dh}$, and the three garbage equations, rewritten in terms of the new witness.

One Iteration of the Main Protocol



One Iteration of the Main Protocol



Instead of sending its last message, the prover outputs the new witness $(\mathbf{s}'_i)_{i\in[r']}$; the verifier outputs the new statement $(\mathcal{F}', \varnothing, \beta')$: $\mathbf{Az} = \sum_i c_i \mathbf{t}_i$, $\mathbf{u}' = \mathbf{Bt} + \mathbf{Cg}$, $\mathbf{u}'' = \mathbf{Dh}$, and the three garbage equations, rewritten in terms of the new witness.

How Fast Does the Witness Shrink?

Count ring elements, ignoring small constants:

- before: $N = rn$;
- after: $\approx 2n$ (for $\mathbf{z}_{\text{low}}, \mathbf{z}_{\text{high}}$) + $\approx r^2$ (for $\mathbf{o}$, dominated by garbage).

How Fast Does the Witness Shrink?

Count ring elements, ignoring small constants:

- before: $N = rn$;
- after: $\approx 2n$ (for $\mathbf{z}_{\text{low}}, \mathbf{z}_{\text{high}}$) + $\approx r^2$ (for $\mathbf{o}$, dominated by garbage).

Minimizing $2N/r + r^2$ gives $r \approx N^{1/3}$ and a new witness of size $\approx N^{2/3}$.

How Fast Does the Witness Shrink?

Count ring elements, ignoring small constants:

- before: $N = rn$;
- after: $\approx 2n$ (for $\mathbf{z}_{\text{low}}, \mathbf{z}_{\text{high}}$) + $\approx r^2$ (for $\mathbf{o}$, dominated by garbage).

Minimizing $2N/r + r^2$ gives $r \approx N^{1/3}$ and a new witness of size $\approx N^{2/3}$.

$$N \to N^{2/3} \to N^{4/9} \to \ldots \quad \Rightarrow \quad O(\log \log N) \text{ iterations}$$

In practice, six or seven iterations; then the prover sends the remaining witness in the clear.

How Fast Does the Witness Shrink?

Count ring elements, ignoring small constants:

- before: $N = rn$;
- after: $\approx 2n$ (for $\mathbf{z}_{\text{low}}$, $\mathbf{z}_{\text{high}}$) + $\approx r^2$ (for $\mathbf{o}$, dominated by garbage).

Minimizing $2N/r + r^2$ gives $r \approx N^{1/3}$ and a new witness of size $\approx N^{2/3}$.

$$N \to N^{2/3} \to N^{4/9} \to \ldots \quad \Rightarrow \quad O(\log \log N) \text{ iterations}$$

In practice, six or seven iterations; then the prover sends the remaining witness in the clear.

- Each iteration contributes only π: $\mathbf{u}', \mathbf{u}''$ ($\kappa' + \kappa''$ ring elements), $\mathbf{p}$ (256 small integers), K_{ct} polynomials v_k: a few kilobytes, independently of the witness size.

The Recursive Protocol

$\mathcal{P}(\mathbb{x}_0, \mathbb{w}_0)$:

1: **for** $i \in [\ell]$:
2: run one iteration on $(\mathbb{x}_i, \mathbb{w}_i)$;
3: $\mathbb{w}_{i+1} \leftarrow$ its output;
4: compute $\mathbb{x}_{i+1}$ as the verifier does;
5: *// last iteration*
send $\mathbb{w}_\ell$.

$\mathcal{V}(\mathbb{x}_0) \to \{0, 1\}$:

1: **for** $i \in [\ell]$:
2: run one iteration on $\mathbb{x}_i$;
3: $\mathbb{x}_{i+1} \leftarrow$ its output;
4: **receive** $\mathbb{w}_\ell$;
5: **return** $[\![(\mathbb{x}_\ell, \mathbb{w}_\ell) \in \mathcal{R}_{\mathsf{dot}}]\!]$.

The Recursive Protocol

$\mathcal{P}(\mathbb{x}_0, \mathbb{w}_0)$:

1: **for** $i \in [\ell]$:
2: run one iteration on $(\mathbb{x}_i, \mathbb{w}_i)$;
3: $\mathbb{w}_{i+1} \leftarrow$ its output;
4: compute $\mathbb{x}_{i+1}$ as the verifier does;
5: *// last iteration*
send $\mathbb{w}_\ell$.

$\mathcal{V}(\mathbb{x}_0) \to \{0, 1\}$:

1: **for** $i \in [\ell]$:
2: run one iteration on $\mathbb{x}_i$;
3: $\mathbb{x}_{i+1} \leftarrow$ its output;
4: **receive** $\mathbb{w}_\ell$;
5: **return** $[\![(\mathbb{x}_\ell, \mathbb{w}_\ell) \in \mathcal{R}_{\mathsf{dot}}]\!]$.

If any check inside an iteration fails, the verifier rejects. The proof consists of the short messages π of all iterations and the final witness $\mathbb{w}_\ell$.

The Recursive Protocol

$\mathcal{P}(\mathbb{x}_0, \mathbb{w}_0)$:

1: **for** $i \in [\ell]$:
2: run one iteration on $(\mathbb{x}_i, \mathbb{w}_i)$;
3: $\mathbb{w}_{i+1} \leftarrow$ its output;
4: compute $\mathbb{x}_{i+1}$ as the verifier does;
5: *// last iteration*
send $\mathbb{w}_\ell$.

$\mathcal{V}(\mathbb{x}_0) \to \{0, 1\}$:

1: **for** $i \in [\ell]$:
2: run one iteration on $\mathbb{x}_i$;
3: $\mathbb{x}_{i+1} \leftarrow$ its output;
4: **receive** $\mathbb{w}_\ell$;
5: **return** $[\![(\mathbb{x}_\ell, \mathbb{w}_\ell) \in \mathcal{R}_{\text{dot}}]\!]$.

If any check inside an iteration fails, the verifier rejects. The proof consists of the short messages π of all iterations and the final witness $\mathbb{w}_\ell$.

Question #4

Suppose the witness has $N = 2^{21}$ ring elements. Show that $r \approx N^{1/3}$ minimizes $2N/r + r^2$, and compute the size of the new witness.

Parameters and Security

Norm Bounds

The honest prover must pass $\sum_i \|\mathbf{s}'_i\|_2^2 \leq \beta'^2$. LaBRADOR models all coefficients as independent Gaussians (matches experiments).

Norm Bounds

The honest prover must pass $\sum_i \|\mathbf{s}'_i\|_2^2 \leq \beta'^2$. LaBRADOR models all coefficients as independent Gaussians (matches experiments).

- Coefficients of $\mathbf{s}_i$: standard deviation $\sigma = \beta/\sqrt{rnd}$. Each coefficient of $\mathbf{z}$: $\approx \sigma\sqrt{r\tau}$, so

$$\|\mathbf{z}\|_2 \approx \sigma\sqrt{r\tau} \cdot \sqrt{nd} = \beta\sqrt{\tau} =: \gamma.$$

Norm Bounds

The honest prover must pass $\sum_i \|\mathbf{s}'_i\|_2^2 \leq \beta'^2$. LaBRADOR models all coefficients as independent Gaussians (matches experiments).

- Coefficients of $\mathbf{s}_i$: standard deviation $\sigma = \beta/\sqrt{rnd}$. Each coefficient of $\mathbf{z}$: $\approx \sigma\sqrt{r\tau}$, so

$$\|\mathbf{z}\|_2 \approx \sigma\sqrt{r\tau} \cdot \sqrt{nd} = \beta\sqrt{\tau} =: \gamma.$$

- $\mathbf{z}_{\text{low}}$ is essentially uniform modulo δ'' (std. dev. $\delta''/\sqrt{12}$), $\mathbf{z}_{\text{high}}$ has $\sigma\sqrt{r\tau}/\delta''$. Equal for

$$\delta'' \approx \sqrt{\sqrt{12r\tau} \cdot \sigma}.$$

Norm Bounds

The honest prover must pass $\sum_i \|\mathbf{s}'_i\|_2^2 \leq \beta'^2$. LaBRADOR models all coefficients as independent Gaussians (matches experiments).

- Coefficients of $\mathbf{s}_i$: standard deviation $\sigma = \beta/\sqrt{rnd}$. Each coefficient of $\mathbf{z}$: $\approx \sigma\sqrt{r\tau}$, so

$$\|\mathbf{z}\|_2 \approx \sigma\sqrt{r\tau} \cdot \sqrt{nd} = \beta\sqrt{\tau} =: \gamma.$$

- $\mathbf{z}_{\text{low}}$ is essentially uniform modulo δ'' (std. dev. $\delta''/\sqrt{12}$), $\mathbf{z}_{\text{high}}$ has $\sigma\sqrt{r\tau}/\delta''$. Equal for

$$\delta'' \approx \sqrt{\sqrt{12r\tau} \cdot \sigma}.$$

- New bound: $\beta'^2 = \frac{2}{\delta''^2}\gamma^2 + \gamma'^2 + \gamma''^2$ (γ', γ'' bound the openings of $\mathbf{u}'$, $\mathbf{u}''$).

Norm Bounds

The honest prover must pass $\sum_i \|\mathbf{s}'_i\|_2^2 \leq \beta'^2$. LaBRADOR models all coefficients as independent Gaussians (matches experiments).

- Coefficients of $\mathbf{s}_i$: standard deviation $\sigma = \beta/\sqrt{rnd}$. Each coefficient of $\mathbf{z}$: $\approx \sigma\sqrt{r\tau}$, so

$$\|\mathbf{z}\|_2 \approx \sigma\sqrt{r\tau} \cdot \sqrt{nd} = \beta\sqrt{\tau} =: \gamma.$$

- $\mathbf{z}_{\text{low}}$ is essentially uniform modulo δ'' (std. dev. $\delta''/\sqrt{12}$), $\mathbf{z}_{\text{high}}$ has $\sigma\sqrt{r\tau}/\delta''$. Equal for

$$\delta'' \approx \sqrt{\sqrt{12r\tau} \cdot \sigma}.$$

- New bound: $\beta'^2 = \frac{2}{\delta''^2}\gamma^2 + \gamma'^2 + \gamma''^2$ (γ', γ'' bound the openings of $\mathbf{u}'$, $\mathbf{u}''$).

Longer-than-predicted honest vectors $\Rightarrow$ restart (running time, not soundness).

Concrete Parameters

- $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^{64} + 1)$ $(d = 64)$, $q \approx 2^{32}$ with $T^{64} + 1$ splitting into two irreducible factors.

Concrete Parameters

- $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^{64} + 1)$ $(d = 64)$, $q \approx 2^{32}$ with $T^{64} + 1$ splitting into two irreducible factors.
- Challenge set with $\tau = 71$ and $\theta = 15$.

Concrete Parameters

- $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^{64}+1)$ $(d = 64)$, $q \approx 2^{32}$ with $T^{64}+1$ splitting into two irreducible factors.
- Challenge set with $\tau = 71$ and $\theta = 15$.
- Commitment ranks κ, κ', κ'' chosen per iteration for 128 bits of Module-SIS security.

Concrete Parameters

- $\mathcal{R}_q = \mathbb{Z}_q[T]/(T^{64}+1)$ $(d = 64)$, $q \approx 2^{32}$ with $T^{64}+1$ splitting into two irreducible factors.
- Challenge set with $\tau = 71$ and $\theta = 15$.
- Commitment ranks κ, κ', κ'' chosen per iteration for 128 bits of Module-SIS security.
- Non-interactive via Fiat-Shamir; all challenges (including the projection matrices) are expanded from short seeds.

Reduction From R1CS

R1CS

Recall: $\mathbf{w}$ satisfies $(\mathbf{L}, \mathbf{R}, \mathbf{O})$ if

$$\mathbf{Lw} \odot \mathbf{Rw} = \mathbf{Ow}, \quad \mathbf{L}, \mathbf{R}, \mathbf{O} \in \mathbb{Z}_N^{k \times \ell} \quad (k \text{ constraints}, \ell \text{ variables}).$$

LaBRADOR supports $N = 2$ (*binary*: hash functions, bit manipulations) and $N = 2^d + 1$ (integer arithmetic). We discuss the binary case.

R1CS

Recall: $\mathbf{w}$ satisfies $(\mathbf{L}, \mathbf{R}, \mathbf{O})$ if

$$\mathbf{Lw} \odot \mathbf{Rw} = \mathbf{Ow}, \quad \mathbf{L}, \mathbf{R}, \mathbf{O} \in \mathbb{Z}_N^{k \times \ell} \quad (k \text{ constraints}, \ell \text{ variables}).$$

LaBRADOR supports $N = 2$ (*binary*: hash functions, bit manipulations) and $N = 2^d + 1$ (integer arithmetic). We discuss the binary case.

Binary R1CS. For $\mathbf{w} \in \{0, 1\}^\ell$, the prover computes

$$\mathbf{a} := \mathbf{Lw} \bmod 2, \quad \mathbf{b} := \mathbf{Rw} \bmod 2, \quad \mathbf{c} := \mathbf{Ow} \bmod 2,$$

packs all bits into polynomial coefficients and commits:
$\mathbf{t} = \mathbf{A}_{\mathsf{R1CS}}(\mathbf{a} \| \mathbf{b} \| \mathbf{c} \| \mathbf{w})$.

R1CS

Recall: $\mathbf{w}$ satisfies $(\mathbf{L}, \mathbf{R}, \mathbf{O})$ if

$$\mathbf{Lw} \odot \mathbf{Rw} = \mathbf{Ow}, \quad \mathbf{L}, \mathbf{R}, \mathbf{O} \in \mathbb{Z}_N^{k \times \ell} \quad (k \text{ constraints}, \ell \text{ variables}).$$

LaBRADOR supports $N = 2$ (*binary*: hash functions, bit manipulations) and $N = 2^d + 1$ (integer arithmetic). We discuss the binary case.

Binary R1CS. For $\mathbf{w} \in \{0, 1\}^\ell$, the prover computes

$$\mathbf{a} := \mathbf{Lw} \bmod 2, \quad \mathbf{b} := \mathbf{Rw} \bmod 2, \quad \mathbf{c} := \mathbf{Ow} \bmod 2,$$

packs all bits into polynomial coefficients and commits: $\mathbf{t} = \mathbf{A}_{\mathsf{R1CS}}(\mathbf{a} \,\|\, \mathbf{b} \,\|\, \mathbf{c} \,\|\, \mathbf{w})$.

It remains to show:

1: all committed coefficients are bits;
2: $\mathbf{a} \odot \mathbf{b} = \mathbf{c}$;
3: $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$ are computed correctly from $\mathbf{w}$ modulo 2.

Binarity and Products

Binarity. With $\mathbf{1}$ the all-ones vector of polynomials:

$$\mathbf{ct}(\langle \mathbf{a}^*, \mathbf{a} - \mathbf{1} \rangle) = \sum_i a_i(a_i - 1) \bmod q.$$

- $\mathbf{a}^*$ is not a dot product function of $\mathbf{a}$: add $\widetilde{\mathbf{a}} := \mathbf{a}^*$ to the witness; $\widetilde{\mathbf{a}} = \mathbf{a}^*$ is $\mathbb{Z}_q$-linear (permute, flip signs): constraints in $\mathcal{F}_{\mathsf{ct}}$.

Binarity and Products

Binarity. With $\mathbf{1}$ the all-ones vector of polynomials:

$$\mathbf{ct}(\langle \mathbf{a}^*, \mathbf{a} - \mathbf{1} \rangle) = \sum_i a_i(a_i - 1) \bmod q.$$

- $\mathbf{a}^*$ is not a dot product function of $\mathbf{a}$: add $\widetilde{\mathbf{a}} := \mathbf{a}^*$ to the witness; $\widetilde{\mathbf{a}} = \mathbf{a}^*$ is $\mathbb{Z}_q$-linear (permute, flip signs): constraints in $\mathcal{F}_{\mathsf{ct}}$.
- The constraint holds mod q, but we need it over $\mathbb{Z}$: every $a_i(a_i - 1) \geq 0$ and $\sum_i a_i(a_i - 1) \leq 2\|\mathbf{a}\|_2^2$. If the certified squared norm is below q, no wraparound: all a_i are bits.

Binarity and Products

Binarity. With $\mathbf{1}$ the all-ones vector of polynomials:

$$\mathbf{ct}(\langle \mathbf{a}^*, \mathbf{a} - \mathbf{1} \rangle) = \sum_i a_i(a_i - 1) \bmod q.$$

- $\mathbf{a}^*$ is not a dot product function of $\mathbf{a}$: add $\widetilde{\mathbf{a}} := \mathbf{a}^*$ to the witness; $\widetilde{\mathbf{a}} = \mathbf{a}^*$ is $\mathbb{Z}_q$-linear (permute, flip signs): constraints in $\mathcal{F}_{\mathsf{ct}}$.
- The constraint holds mod q, but we need it over $\mathbb{Z}$: every $a_i(a_i - 1) \geq 0$ and $\sum_i a_i(a_i - 1) \leq 2\|\mathbf{a}\|_2^2$. If the certified squared norm is below q, no wraparound: all a_i are bits.

Products. For bits, $ab = c$ iff $a + b - 2c \in \{0, 1\}$: show

$$\mathbf{ct}(\langle \widetilde{\mathbf{a}} + \widetilde{\mathbf{b}} - 2\widetilde{\mathbf{c}}, \mathbf{a} + \mathbf{b} - 2\mathbf{c} - \mathbf{1} \rangle) = 0.$$

Linear Relations Modulo 2

$3k$ relations modulo 2, but the main protocol works modulo q.
Compress them into λ random combinations:
$\alpha_{i,L}, \alpha_{i,R}, \alpha_{i,O} \leftarrow\$ \{0,1\}^k$ for $i \in [\lambda]$, and

$$\delta_i := \alpha_{i,L}^\top \mathbf{L} + \alpha_{i,R}^\top \mathbf{R} + \alpha_{i,O}^\top \mathbf{O} \bmod 2 \in \{0,1\}^\ell.$$

Linear Relations Modulo 2

$3k$ relations modulo 2, but the main protocol works modulo q.
Compress them into λ random combinations:
$\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O} \leftarrow\$ \{0,1\}^k$ for $i \in [\lambda]$, and

$$\boldsymbol{\delta}_i := \boldsymbol{\alpha}_{i,L}^\top \mathbf{L} + \boldsymbol{\alpha}_{i,R}^\top \mathbf{R} + \boldsymbol{\alpha}_{i,O}^\top \mathbf{O} \bmod 2 \in \{0,1\}^\ell.$$

The prover sends the integers (dot products over $\mathbb{Z}$ on the bits)

$$e_i := \langle \boldsymbol{\alpha}_{i,L}, \mathbf{a} \rangle + \langle \boldsymbol{\alpha}_{i,R}, \mathbf{b} \rangle + \langle \boldsymbol{\alpha}_{i,O}, \mathbf{c} \rangle - \langle \boldsymbol{\delta}_i, \mathbf{w} \rangle, \quad i \in [\lambda].$$

Linear Relations Modulo 2

$3k$ relations modulo 2, but the main protocol works modulo q.
Compress them into λ random combinations:
$\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O} \leftarrow\$ \{0,1\}^k$ for $i \in [\lambda]$, and

$$\boldsymbol{\delta}_i := \boldsymbol{\alpha}_{i,L}^\top \mathbf{L} + \boldsymbol{\alpha}_{i,R}^\top \mathbf{R} + \boldsymbol{\alpha}_{i,O}^\top \mathbf{O} \bmod 2 \in \{0,1\}^\ell.$$

The prover sends the integers (dot products over $\mathbb{Z}$ on the bits)

$$e_i := \langle \boldsymbol{\alpha}_{i,L}, \mathbf{a} \rangle + \langle \boldsymbol{\alpha}_{i,R}, \mathbf{b} \rangle + \langle \boldsymbol{\alpha}_{i,O}, \mathbf{c} \rangle - \langle \boldsymbol{\delta}_i, \mathbf{w} \rangle, \quad i \in [\lambda].$$

- All relations hold $\Rightarrow$ every e_i is even (the verifier checks).

Linear Relations Modulo 2

$3k$ relations modulo 2, but the main protocol works modulo q.
Compress them into λ random combinations:
$\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O} \leftarrow\$ \{0,1\}^k$ for $i \in [\lambda]$, and

$$\boldsymbol{\delta}_i := \boldsymbol{\alpha}_{i,L}^\top \mathbf{L} + \boldsymbol{\alpha}_{i,R}^\top \mathbf{R} + \boldsymbol{\alpha}_{i,O}^\top \mathbf{O} \bmod 2 \in \{0,1\}^\ell.$$

The prover sends the integers (dot products over $\mathbb{Z}$ on the bits)

$$e_i := \langle \boldsymbol{\alpha}_{i,L}, \mathbf{a} \rangle + \langle \boldsymbol{\alpha}_{i,R}, \mathbf{b} \rangle + \langle \boldsymbol{\alpha}_{i,O}, \mathbf{c} \rangle - \langle \boldsymbol{\delta}_i, \mathbf{w} \rangle, \quad i \in [\lambda].$$

- All relations hold $\Rightarrow$ every e_i is even (the verifier checks).
- Some relation fails $\Rightarrow$ each e_i is odd w.p. $1/2$: soundness error $2^{-\lambda}$.

Linear Relations Modulo 2

$3k$ relations modulo 2, but the main protocol works modulo q.
Compress them into λ random combinations:
$\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O} \leftarrow\$ \{0,1\}^k$ for $i \in [\lambda]$, and

$$\boldsymbol{\delta}_i := \boldsymbol{\alpha}_{i,L}^\top \mathbf{L} + \boldsymbol{\alpha}_{i,R}^\top \mathbf{R} + \boldsymbol{\alpha}_{i,O}^\top \mathbf{O} \bmod 2 \in \{0,1\}^\ell.$$

The prover sends the integers (dot products over $\mathbb{Z}$ on the bits)

$$e_i := \langle \boldsymbol{\alpha}_{i,L}, \mathbf{a} \rangle + \langle \boldsymbol{\alpha}_{i,R}, \mathbf{b} \rangle + \langle \boldsymbol{\alpha}_{i,O}, \mathbf{c} \rangle - \langle \boldsymbol{\delta}_i, \mathbf{w} \rangle, \quad i \in [\lambda].$$

- All relations hold $\Rightarrow$ every e_i is even (the verifier checks).
- Some relation fails $\Rightarrow$ each e_i is odd w.p. $1/2$: soundness error $2^{-\lambda}$.
- Correctness of e_i: a constraint in $\mathcal{F}_{\mathsf{ct}}$, valid over $\mathbb{Z}$ as long as $\ell + 3k < q$.

The Binary R1CS Reduction

$\mathcal{P}(\mathbb{x} = (\mathbf{L}, \mathbf{R}, \mathbf{O}), \mathbb{w} = \mathbf{w})$:

1: $\mathbf{a}, \mathbf{b}, \mathbf{c} \leftarrow \mathbf{Lw}, \mathbf{Rw}, \mathbf{Ow} \bmod 2$;
2: **send** $\mathbf{t} \leftarrow \mathbf{A}_{\mathsf{R1CS}}(\mathbf{a} \,\|\, \mathbf{b} \,\|\, \mathbf{c} \,\|\, \mathbf{w})$;
3: **receive** $(\alpha_{i,L}, \alpha_{i,R}, \alpha_{i,O})_{i \in [\lambda]}$;
4: compute δ_i and e_i for $i \in [\lambda]$;
5: **send** $(e_i)_{i \in [\lambda]}$;
6: *// target witness*
return $(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{w}, \mathbf{a}^*, \mathbf{b}^*, \mathbf{c}^*, \mathbf{w}^*)$.

$\mathcal{V}(\mathbb{x} = (\mathbf{L}, \mathbf{R}, \mathbf{O}))$:

1: **receive** $\mathbf{t}$;
2: **send** $\alpha_{i,L}, \alpha_{i,R}, \alpha_{i,O} \leftarrow\$ \{0,1\}^k$, $i \in [\lambda]$;
3: **receive** $(e_i)_{i \in [\lambda]}$;
4: **assert** e_i is even for all $i \in [\lambda]$;
5: *// target statement*
return $(\mathcal{F}, \mathcal{F}_{\mathsf{ct}}, \sqrt{2(3k + \ell)})$.

The Binary R1CS Reduction

$\mathcal{P}(\mathbb{x} = (\mathbf{L}, \mathbf{R}, \mathbf{O}), \mathbb{w} = \mathbf{w})$:

1: $\mathbf{a}, \mathbf{b}, \mathbf{c} \leftarrow \mathbf{Lw}, \mathbf{Rw}, \mathbf{Ow} \bmod 2$;
2: **send** $\mathbf{t} \leftarrow \mathbf{A}_{\mathsf{R1CS}}(\mathbf{a} \,\|\, \mathbf{b} \,\|\, \mathbf{c} \,\|\, \mathbf{w})$;
3: **receive** $(\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O})_{i \in [\lambda]}$;
4: compute $\boldsymbol{\delta}_i$ and e_i for $i \in [\lambda]$;
5: **send** $(e_i)_{i \in [\lambda]}$;
6: *// target witness*
return $(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{w}, \mathbf{a}^*, \mathbf{b}^*, \mathbf{c}^*, \mathbf{w}^*)$.

$\mathcal{V}(\mathbb{x} = (\mathbf{L}, \mathbf{R}, \mathbf{O}))$:

1: **receive** $\mathbf{t}$;
2: **send** $\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O} \leftarrow\$ \{0,1\}^k$, $i \in [\lambda]$;
3: **receive** $(e_i)_{i \in [\lambda]}$;
4: **assert** e_i is even for all $i \in [\lambda]$;
5: *// target statement*
return $(\mathcal{F}, \mathcal{F}_{\mathsf{ct}}, \sqrt{2(3k + \ell)})$.

- $\mathcal{F}$: the opening of $\mathbf{t}$; $\mathcal{F}_{\mathsf{ct}}$: conjugation relations, binarity of $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$, $\mathbf{w}$, $\mathbf{a} + \mathbf{b} - 2\mathbf{c}$, and correctness of the e_i.

The Binary R1CS Reduction

$\mathcal{P}(\mathbb{x} = (\mathbf{L}, \mathbf{R}, \mathbf{O}), \mathbb{w} = \mathbf{w})$:

1: $\mathbf{a}, \mathbf{b}, \mathbf{c} \leftarrow \mathbf{Lw}, \mathbf{Rw}, \mathbf{Ow} \bmod 2$;
2: **send** $\mathbf{t} \leftarrow \mathbf{A}_{\mathsf{R1CS}}(\mathbf{a} \,\|\, \mathbf{b} \,\|\, \mathbf{c} \,\|\, \mathbf{w})$;
3: **receive** $(\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O})_{i \in [\lambda]}$;
4: compute $\boldsymbol{\delta}_i$ and e_i for $i \in [\lambda]$;
5: **send** $(e_i)_{i \in [\lambda]}$;
6: *// target witness*
return $(\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{w}, \mathbf{a}^*, \mathbf{b}^*, \mathbf{c}^*, \mathbf{w}^*)$.

$\mathcal{V}(\mathbb{x} = (\mathbf{L}, \mathbf{R}, \mathbf{O}))$:

1: **receive** $\mathbf{t}$;
2: **send** $\boldsymbol{\alpha}_{i,L}, \boldsymbol{\alpha}_{i,R}, \boldsymbol{\alpha}_{i,O} \leftarrow\$ \{0,1\}^k$, $i \in [\lambda]$;
3: **receive** $(e_i)_{i \in [\lambda]}$;
4: **assert** e_i is even for all $i \in [\lambda]$;
5: *// target statement*
return $(\mathcal{F}, \mathcal{F}_{\mathsf{ct}}, \sqrt{2(3k + \ell)})$.

- $\mathcal{F}$: the opening of $\mathbf{t}$; $\mathcal{F}_{\mathsf{ct}}$: conjugation relations, binarity of $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$, $\mathbf{w}$, $\mathbf{a} + \mathbf{b} - 2\mathbf{c}$, and correctness of the e_i.
- Squared norm of the target witness $\leq 2(3k + \ell)$; $2(3k + \ell) \cdot \frac{128}{30} < q$ holds for all practical sizes with $q \approx 2^{32}$.

Summary

- LaBRADOR proves **dot product constraint systems** $\mathcal{R}_{\text{dot}}$: quadratic constraints over $\mathcal{R}_q$, constant-coefficient constraints, and a norm bound.

Summary

- LaBRADOR proves **dot product constraint systems** $\mathcal{R}_{\text{dot}}$: quadratic constraints over $\mathcal{R}_q$, constant-coefficient constraints, and a norm bound.
- Main protocol: **amortized opening** $\mathbf{z} = \sum_i c_i \mathbf{s}_i$, **garbage polynomials**, **aggregation** of all constraints into one, and a **JL projection** for the norm (slack ≈ 2.07).

Summary

- LaBRADOR proves **dot product constraint systems** $\mathcal{R}_{\text{dot}}$: quadratic constraints over $\mathcal{R}_q$, constant-coefficient constraints, and a norm bound.
- Main protocol: **amortized opening** $\mathbf{z} = \sum_i c_i \mathbf{s}_i$, **garbage polynomials**, **aggregation** of all constraints into one, and a **JL projection** for the norm (slack ≈ 2.07).
- No zero-knowledge: no masks, no rejection sampling.

Summary

- LaBRADOR proves **dot product constraint systems** $\mathcal{R}_{\text{dot}}$: quadratic constraints over $\mathcal{R}_q$, constant-coefficient constraints, and a norm bound.
- Main protocol: **amortized opening** $\mathbf{z} = \sum_i c_i \mathbf{s}_i$, **garbage polynomials**, **aggregation** of all constraints into one, and a **JL projection** for the norm (slack ≈ 2.07).
- No zero-knowledge: no masks, no rejection sampling.
- The verifier's checks are again dot product constraints: **recursion** via outer commitments, decomposition and reshaping; the witness shrinks $N \to N^{2/3}$, so $O(\log \log N)$ iterations.

Summary

- LaBRADOR proves **dot product constraint systems** $\mathcal{R}_{\text{dot}}$: quadratic constraints over $\mathcal{R}_q$, constant-coefficient constraints, and a norm bound.
- Main protocol: **amortized opening** $\mathbf{z} = \sum_i c_i \mathbf{s}_i$, **garbage polynomials**, **aggregation** of all constraints into one, and a **JL projection** for the norm (slack ≈ 2.07).
- No zero-knowledge: no masks, no rejection sampling.
- The verifier's checks are again dot product constraints: **recursion** via outer commitments, decomposition and reshaping; the witness shrinks $N \to N^{2/3}$, so $O(\log \log N)$ iterations.
- Formally: a chain of **reductions of knowledge** that compose sequentially and in parallel.

Summary

- LaBRADOR proves **dot product constraint systems** $\mathcal{R}_{\text{dot}}$: quadratic constraints over $\mathcal{R}_q$, constant-coefficient constraints, and a norm bound.
- Main protocol: **amortized opening** $\mathbf{z} = \sum_i c_i \mathbf{s}_i$, **garbage polynomials**, **aggregation** of all constraints into one, and a **JL projection** for the norm (slack ≈ 2.07).
- No zero-knowledge: no masks, no rejection sampling.
- The verifier's checks are again dot product constraints: **recursion** via outer commitments, decomposition and reshaping; the witness shrinks $N \to N^{2/3}$, so $O(\log \log N)$ iterations.
- Formally: a chain of **reductions of knowledge** that compose sequentially and in parallel.
- Binary R1CS reduces to $\mathcal{R}_{\text{dot}}$; 2^{20} constraints $\to \approx 58$ KB, from Module-SIS only.

Thank you for your attention



research.blockstream.com

github.com/BlockstreamResearch/zk-book



Blockstream